Technical Design

Battle Engine

System Design, Exam Assignment 1 (TD 1)

Rokas Pazera (551495)
HBO-ICT
Version 1.1, 8 October 2026

Contents

1. Notation 4
2. Goal 5
3. Requirements and interpretations 6
3.1 Interface 6
3.2 Functional requirements 6
3.3 Interpretations and assumptions 7
4. C4 Layer 1: Context 8
4.1 System context diagram 8
4.2 Use case diagram: Online Game System 8
4.3 Use case diagram: the Battle Engine in its context 9
4.4 Design notes 10
5. C4 Layer 2: Container 11
5.1 Container diagram 11
5.2 Deployment diagram 12
5.3 Component diagram 13
5.4 Instance life cycle 15
5.5 Design notes 16
6. C4 Layer 3: Component 17
6.1 Component diagram 17
6.2 Component diagram (UML) 18
6.3 Deployment diagram (detailed) 19
6.4 Dynamic view 21
6.5 Design notes 23
7. C4 Layer 4: Code 25
7.1 Package overview 25
7.2 Interface and facade 27
7.3 Battle model 29
7.4 Actions and effects 31
7.5 Validation 33
7.6 Configuration and factories 34
7.7 Battle log 35
7.8 Life cycle of a battle 37
7.9 applyAction step by step 38
7.10 Implementation of the rules 39
8. SOLID principles and design patterns 41

8.1 SOLID principles 41
8.2 Design patterns 42
8.3 Extensibility scenarios 42
9. Requirements traceability 43
10. Design Justification 44
10.1 Design process 44
10.2 Design decisions 45
10.3 Insights 45

1. Notation

- **C4 diagrams** are made with C4-PlantUML. Dark blue is a person, blue an element inside the scope of the diagram, grey an external element and purple the element that contains the Battle Engine. Every C4 diagram has a legend, and every relationship says what happens and, where useful, which technology is used.
- **UML diagrams** follow UML 2.5: use case diagrams with a system boundary, «include» and «extend»; deployment diagrams with «device», «executionEnvironment», «artifact» and «deployment spec»; component diagrams with provided interfaces (ball) and required interfaces (socket); class diagrams with visibility (+ public, - private), multiplicities, constraints in braces and notes; sequence and state machine diagrams for behaviour. An attribute marked {readOnly} is a private field with a getter and no setter.
- **Names** follow Java conventions: classes and interfaces in PascalCase without an "I" prefix, methods and attributes in camelCase, packages in lower case. The six methods of the assignment therefore start with a lower-case letter (StartBattle becomes `startBattle`). Interfaces between containers end with "Api" (for example PlayerDataApi).

2. Goal

The Battle Engine runs a single turn-based battle between two players: the human player, who controls 1 to 4 characters, and the computer, which controls 1 to 4 enemies. During a battle it keeps the complete state: which player is active, the HP and stats of every unit and which units are knocked out. It checks whether a requested action is allowed, applies its effects (damage, healing or a stat change), removes knocked-out units, decides when the battle is over and records everything in a battle log.

The Battle Engine is a rules engine and nothing more. It does not know how the player presses a button, how the computer decides what to do, what the screen looks like or where data is stored. Other parts of the game only talk to it through the six methods of its interface: `startBattle`, `getBattleState`, `getAvailableActions`, `applyAction`, `nextTurn` and `getBattleLog`. Everything it needs comes in through `ConfigBattle`; everything others need to know comes out through `getBattleState()` and `getBattleLog()`.

The game is always played online. Every battle runs inside a game instance on a server, and an instance is stopped when no players are left on it. Therefore the Battle Engine keeps a battle only in memory and has no storage of its own. Results that must be kept, such as experience and levels, are saved by another component of the game instance through the Persistence Service (chapter 6).

Quality goal	What it means for the design
Correctness	Every rule of the assignment is implemented in exactly one place and can be tested without a server or a network.
Server authority	The client only says which action to use on which targets. Damage, healing and stat values always come from the engine's own data, so a modified client cannot cheat.
Encapsulation	Nobody can change a running battle except through `applyAction` and `nextTurn`.
Extensibility	New action types, rules and controllers can be added without changing existing engine classes.
Independence	The engine depends on no other part of the game, so it can be reused, for example in a test tool or a later PvP mode.

3. Requirements and interpretations

This chapter gives every requirement of the assignment an ID, so that the design can refer to it and chapter 9 can show where each one is implemented.

3.1 Interface

The method names are written in Java style (camelCase); parameters and return types are unchanged.

ID	Method	Parameters and return type (my definition)
R-IF1	`void startBattle(ConfigBattle config)`	ConfigBattle: battle id, the player that starts and 1-4 units per player with HP, stats and actions (section 7.6).
R-IF2	`Battle getBattleState()`	Battle: a complete, detached copy of the current battle (section 7.3).
R-IF3	`List<Action> getAvailableActions(int unitId)`	One Action per action the unit can use now, with the units it may target (section 7.4).
R-IF4	`void applyAction(int unitId, Action action)`	Action with actionId, casterId and the chosen targetIds (section 7.4).
R-IF5	`void nextTurn()`	No parameters.
R-IF6	`BattleLog getBattleLog(UUID battleId)`	BattleLog with ordered entries; `toText()` gives lines like the example in the assignment (section 7.7).

3.2 Functional requirements

ID	Requirement
R-T1	The engine always has exactly one active player.
R-T2	There are two players: Human and Computer.
R-T3	A system outside the engine (player input or AI) calls applyAction.
R-T4	nextTurn makes the other player the active player.
R-T5	Only units of the active player can perform actions.
R-A1	The unitId of applyAction must belong to the active player; that unit performs the action.
R-A2	An action has one caster and 1..* other units as targets; this data is in the Action object.
R-A3	Action type: the targets take X damage.
R-A4	Action type: the targets are healed for X.
R-A5	Action type: a stat of the targets goes up or down by X%, rounded up.
R-A6	For a stat change the stats of the caster do not matter.
R-U1	Every unit has HP; damage lowers it, healing raises it.
R-U2	A unit with 0 HP or less is knocked out and removed: it can no longer act or be targeted.
R-U3	HP never exceeds max HP.
R-U4	Damage = base damage of the action + caster's Power - target's Defence.
R-U5	Healing = base healing of the action + caster's Spirit.
R-S1	startBattle provides the state to start a battle; getBattleState returns the complete current state.
R-L1	getBattleLog returns the log of the battle, like the example in the assignment.
R-C1	The game is always online; a player always plays on one instance.
R-C2	Playing alone starts a new private instance.
R-C3	A player can visit another player's world by switching to that player's instance.
R-C4	An instance without players is shut down.
R-C5	An instance has at most 8 players.
R-C6	No permanent data is stored on an instance; a separate component stores it.
R-C7	The player installs a client application on their own computer.
R-C8	A battle has 1-4 player characters and 1-4 enemies; always the player against the computer, never between players and never with players together.

3.3 Interpretations and assumptions

The assignment leaves some points open. These are my interpretations; none of them contradicts or overrides a requirement.

ID	Open point	Interpretation	Reason
I-1	Which player starts?	`ConfigBattle.firstPlayer`, default Human.	The example log starts with player 1 (Human); an ambush (computer first) is possible without code changes.
I-2	When does a battle end?	When all units of one player are knocked out; the other player wins. After that applyAction and nextTurn are refused.	The assignment defines no end; this is the usual rule in turn-based RPGs such as Dragon Quest XI.
I-3	How many actions per unit per turn?	No limit: the turn ends when nextTurn() is called.	The assignment sets no limit and the example log shows unit A acting twice in one turn. A limit can be added as an extra rule (section 8.3).
I-4	Can a unit target itself?	No.	R-A2 speaks of "1..* other units".
I-5	Which units can be targeted?	Every action has a TargetType (ENEMY, ALLY or ANY) and a maximum number of targets.	Otherwise every action could hit every unit. It narrows "1..*" down per action and does not change it.
I-6	Negative damage	If Power - Defence makes the damage negative, the damage is 0.	Otherwise a high Defence would heal the target, which contradicts "Defence reduces the damage".
I-7	Rounding of stat changes	Always up (ceiling), also for decreases; a stat never goes below 0.	"round up" in R-A5; example 1 × 1.25 = 2.
I-8	HP below 0	HP is stored as 0, never negative. The log shows the calculated damage.	Keeps 0 ≤ HP ≤ max HP true at all times.
I-9	What does "removed" mean?	A knocked-out unit moves from the active units of its player to the knocked-out units. It no longer takes part, but stays visible in the state and the log.	Meets R-U2 and keeps the state complete (R-S1).
I-10	Several battles on one instance	One BattleEngineFacade object per battle.	Up to 8 players on one instance can each be in a battle, while nextTurn() and getBattleState() have no battleId (decision D3).
I-11	Login	Players log in with a studio-wide account at an external Account Provider.	The case does not describe login, but player data must belong to a known player.
I-12	Leaving during a battle	A battle is abandoned when the player disconnects; nothing from it is saved. Switching instances is not possible during a battle.	No half-finished battle has to be stored (R-C6).

4. C4 Layer 1: Context

4.1 System context diagram

At this layer the whole game is one software system, the Online Game System. Its only user is the Player. The Account Provider is the only external system: the studio's existing account system, which is used to log in (I-11). Instances, persistence and the Battle Engine are all inside the Online Game System and are shown in layer 2 and deeper.

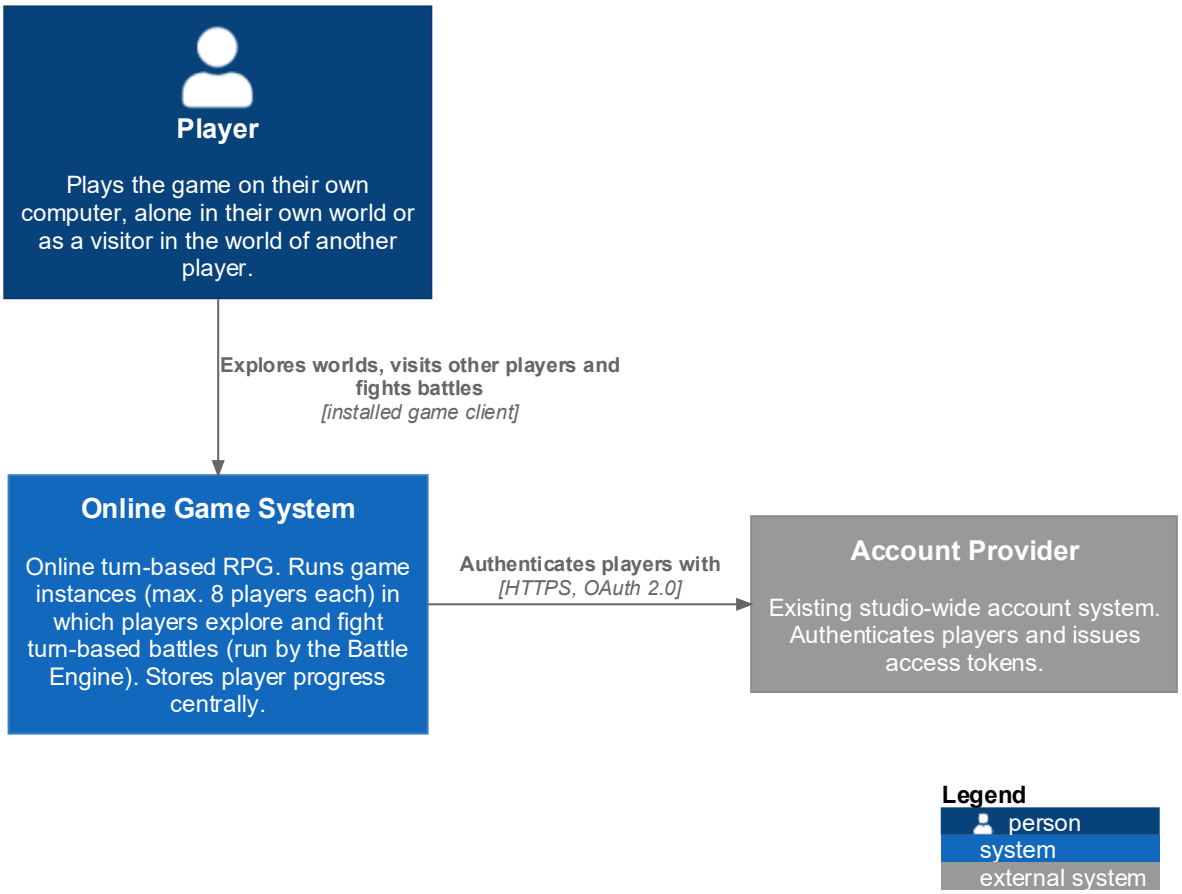


Figure 4.1: C4 system context diagram

4.2 Use case diagram: Online Game System

Figure 4.2 shows what the Player can do with the system. Play in own world and Visit another player's world are the two ways to get onto an instance (R-C2, R-C3). Fight turn-based battle extends Explore the world at the extension point "enemy encountered": as in Dragon Quest XI, a battle starts when the player runs into enemies while exploring.

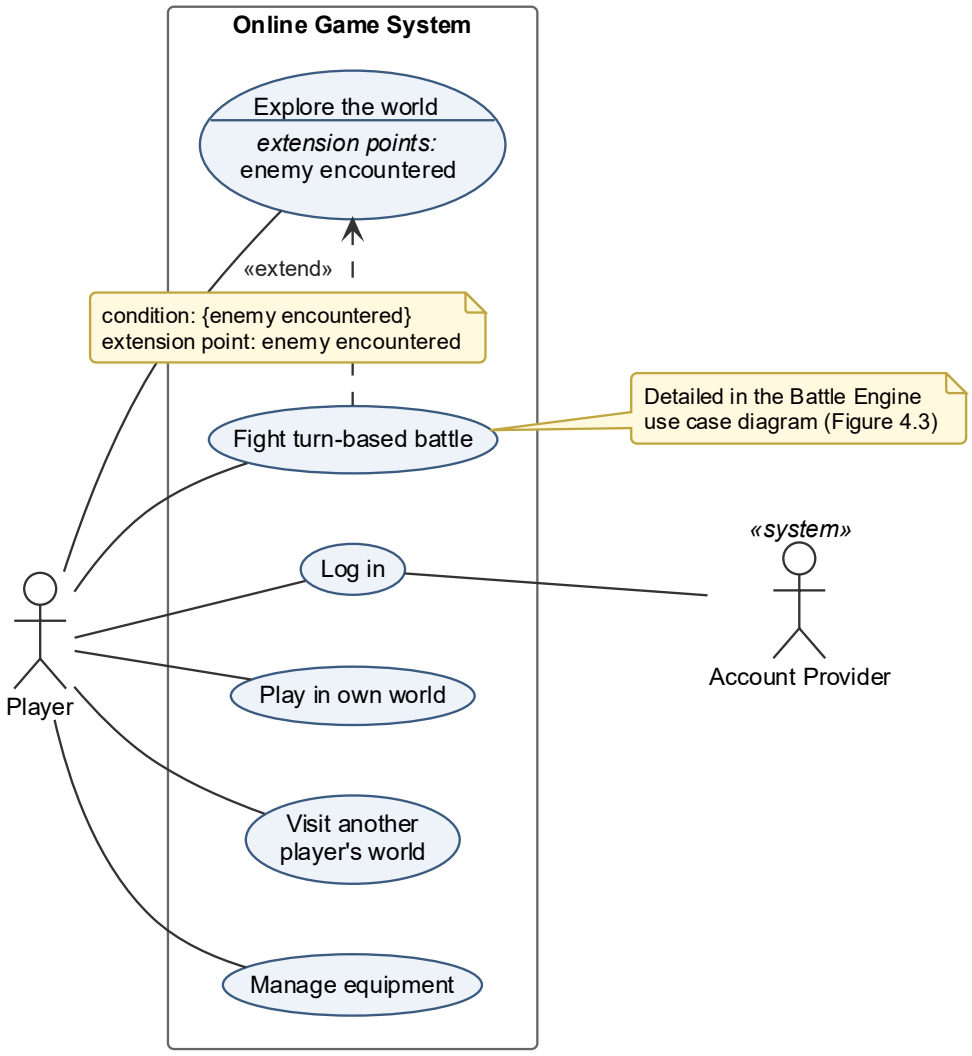


Figure 4.2: UML use case diagram of the Online Game System

4.3 Use case diagram: the Battle Engine in its context

To show the context in which the Battle Engine is used, Figure 4.3 takes the Battle Engine itself as the subject. From its point of view the other parts of the game instance are actors, and its six interface methods are the six main use cases. Battle Controller is an abstract actor (name in italics): the Human Battle Controller (player input) and the AI Battle Controller (computer) are specialisations of it, because they use the engine in the same way. Apply action always includes Validate action and Resolve effects; Knock out unit and End battle only happen under a condition, so they extend Apply action.

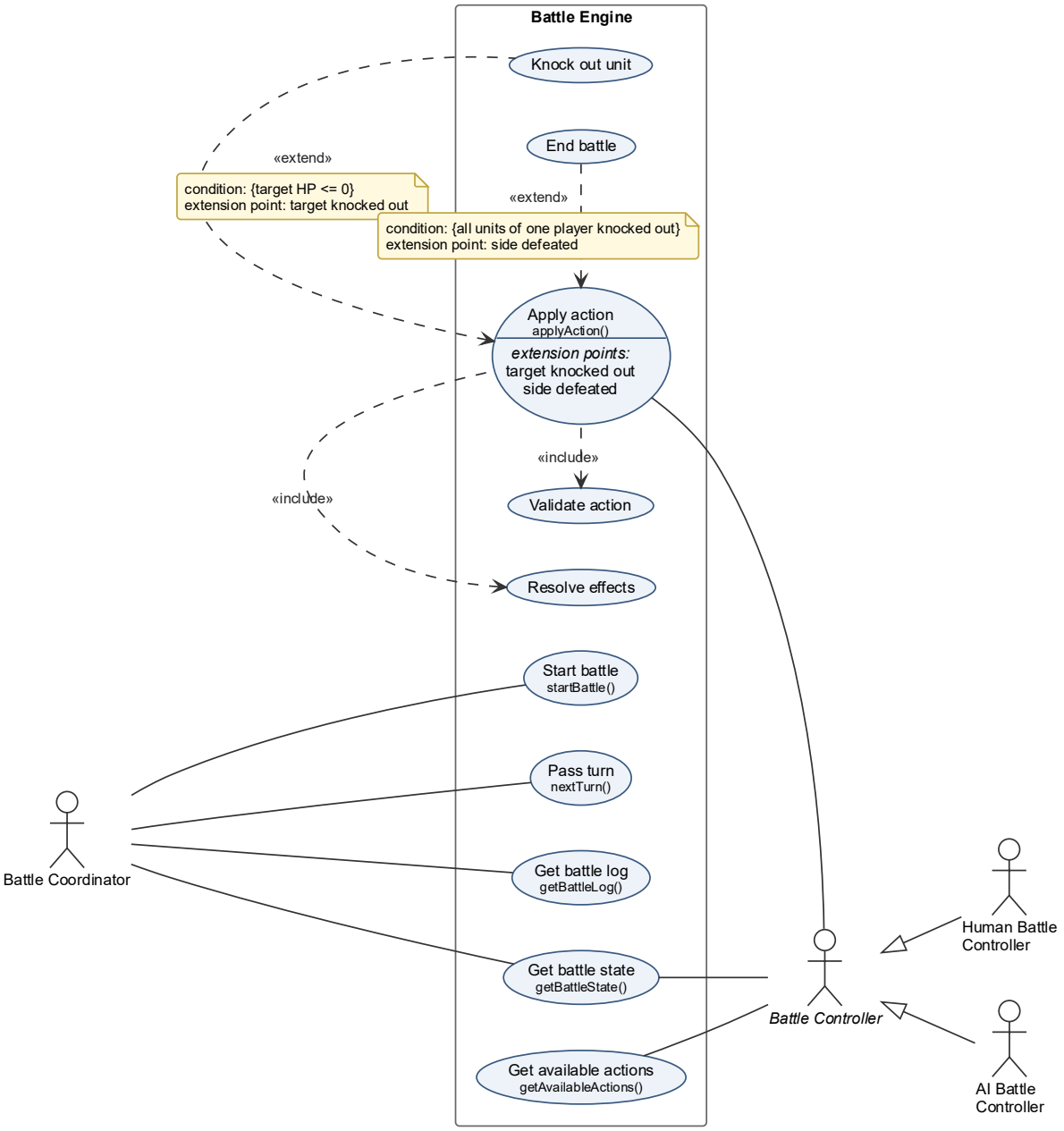


Figure 4.3: UML use case diagram of the Battle Engine

4.4 Design notes

- **DN-1.1 The computer opponent is not an actor in Figure 4.2.** The assignment says the AI system is "outside the battle engine". That means outside the Battle Engine part, not outside the game: the AI runs on our own servers. At system level it is inside the system, and Figure 4.2 only shows actors outside the whole system. In Figure 4.3, where the Battle Engine is the subject, the AI is an actor.
- **DN-1.2 The Account Provider is external,** because login is shared with the other games of the studio and is not part of this assignment.
- **DN-1.3 Switching instances is not a use case.** The goal of the player is to visit a world; switching instances is how the system achieves that (section 5.4).

5. C4 Layer 2: Container

5.1 Container diagram

Figure 5.1 zooms in on the Online Game System. It consists of five containers. The Game Instance (purple) is the container that runs the Battle Engine; chapter 6 zooms in on it.

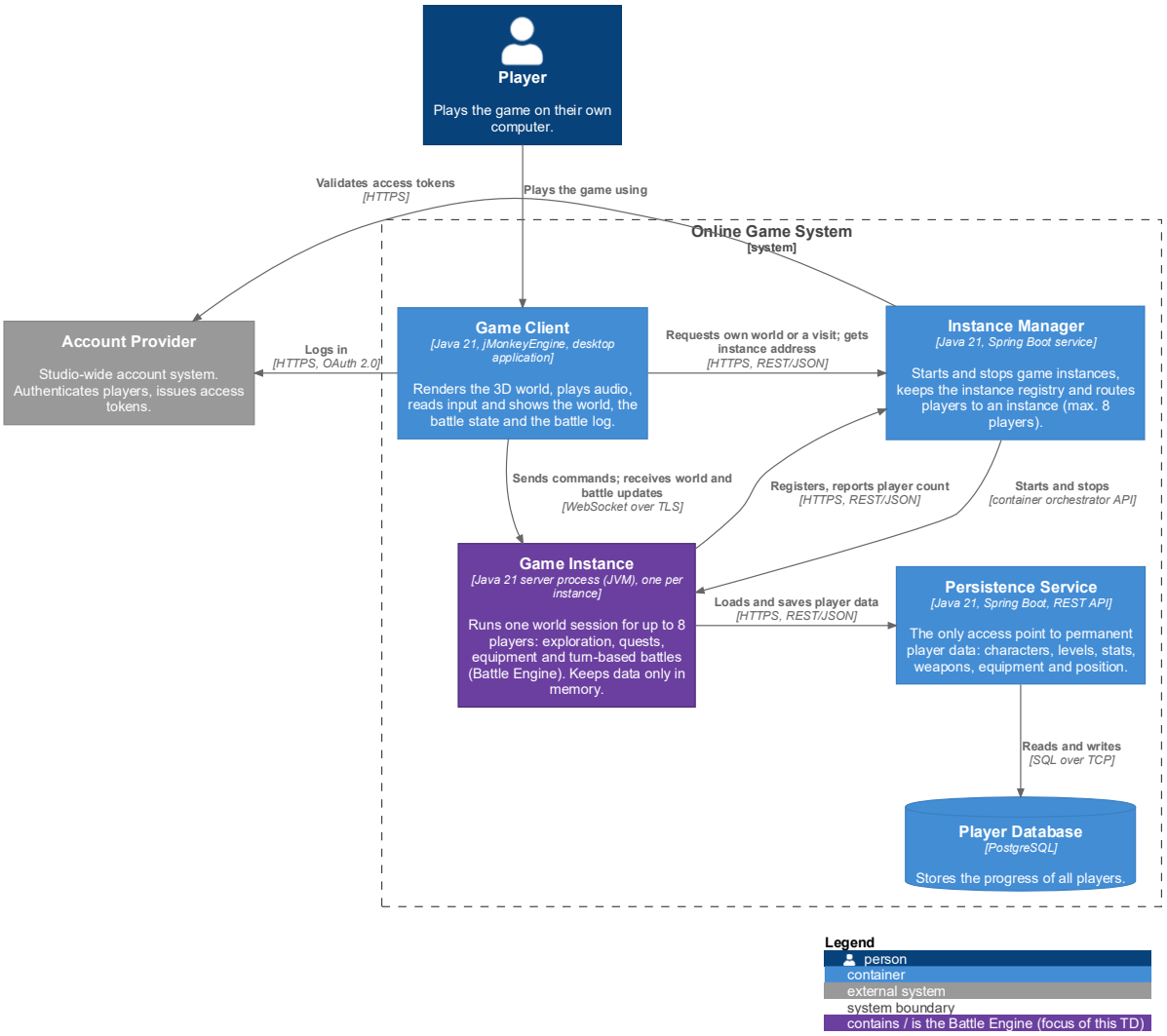


Figure 5.1: C4 container diagram

Container	Technology	Responsibility	Why a separate container
Game Client	Java 21, jMonkeyEngine, desktop application	Shows the world and battles (3D models, audio), reads input, sends commands.	The case requires an installed client (R-C7); rendering belongs on the player's computer.
Instance Manager	Java 21, Spring Boot service	Starts private instances, routes visitors, enforces the maximum of 8 players, stops empty instances, keeps the instance registry.	Instances come and go; a part that lives longer must know which instances exist (R-C2 to R-C5).
Game Instance	Java 21 server process in a container, one per instance	Runs one world session for up to 8 players: exploration, quests, equipment and battles. Keeps data only in memory.	Every instance must be startable and stoppable on its own (R-C1, R-C4).
Persistence Service	Java 21, Spring Boot REST API	The only access point to permanent player data.	No permanent data on an instance (R-C6); every instance must see the same, current data.
Player Database	PostgreSQL	Stores characters, levels, stats, weapons, equipment and world position.	Relational data (player, characters, items) that needs transactions.

Technology choice. All containers use Java 21, the long-term support version of Java that is also used in this course. The client uses jMonkeyEngine, a 3D game engine for Java, and is installed with an installer that includes the Java runtime, so the player does not have to install Java. The services use Spring Boot. With one language the client and the server share the same data classes (for example Battle and Action) for the network messages (decision D13).

5.2 Deployment diagram

Figure 5.2 shows on which hardware the containers run. The Game Servers are virtual machines in the cloud that are added and removed automatically when the number of instances changes ({1..*, auto-scaled}). Every instance runs in its own container ({0..*, one per instance}), so a crashing instance cannot affect another one and an instance is stopped by removing its container.

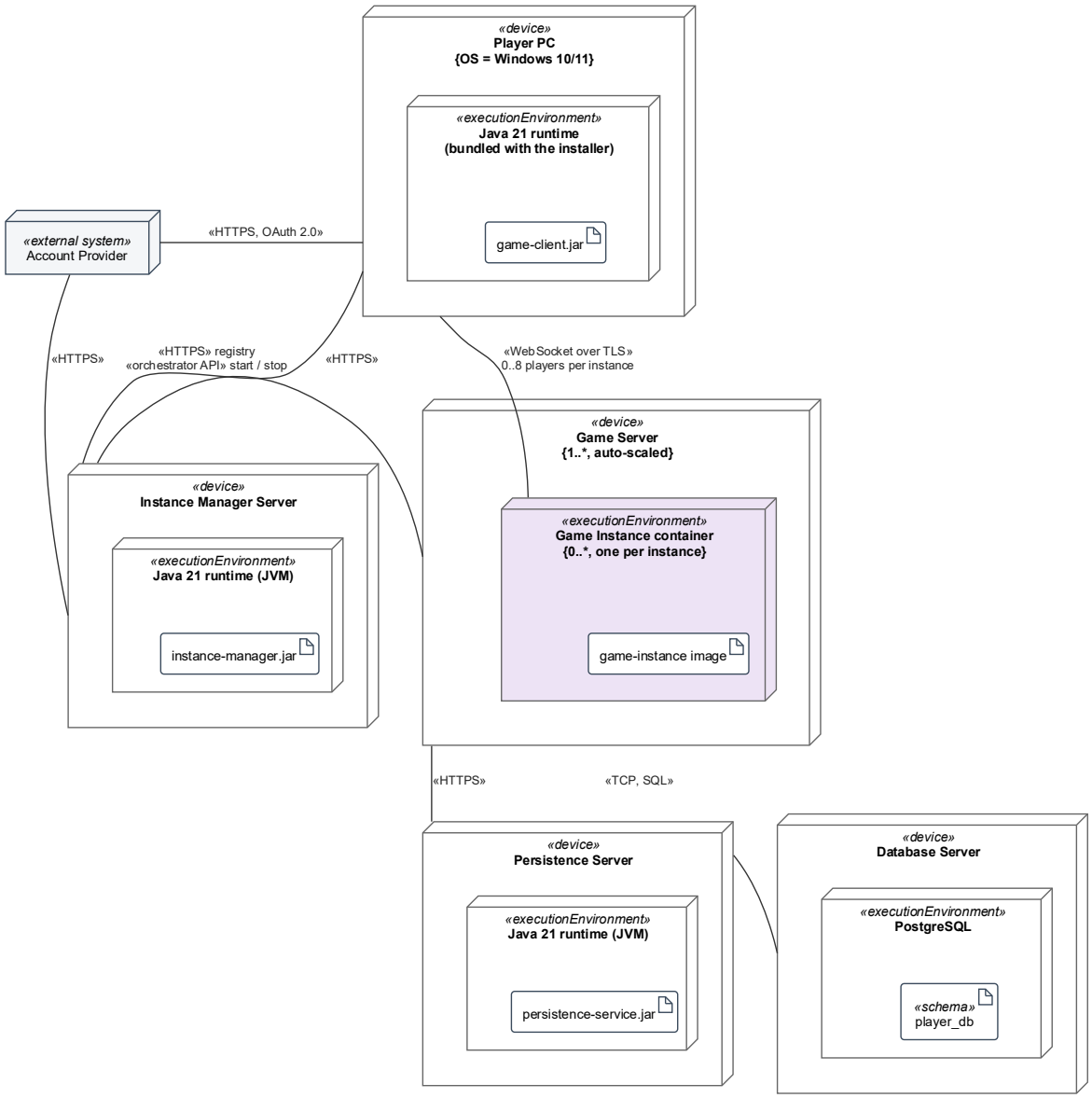


Figure 5.2: UML deployment diagram (layer 2)

- **WebSocket between client and instance.** One long-lived connection in both directions: the server can push updates (other players moving, the computer's battle actions) without the client asking for them. A turn-based battle does not need the lower latency of UDP, and WebSocket over TLS works through firewalls.
- **HTTPS with REST for the services.** Requests to the Instance Manager and the Persistence Service are short request and response calls; REST with JSON is simple and easy to monitor.
- **Database on its own server.** The database is the only place with permanent data, so it gets its own machine with backups, separate from the stateless services.

5.3 Component diagram

Figure 5.3 shows the same containers as UML components with the interfaces they provide (ball) and require (socket).

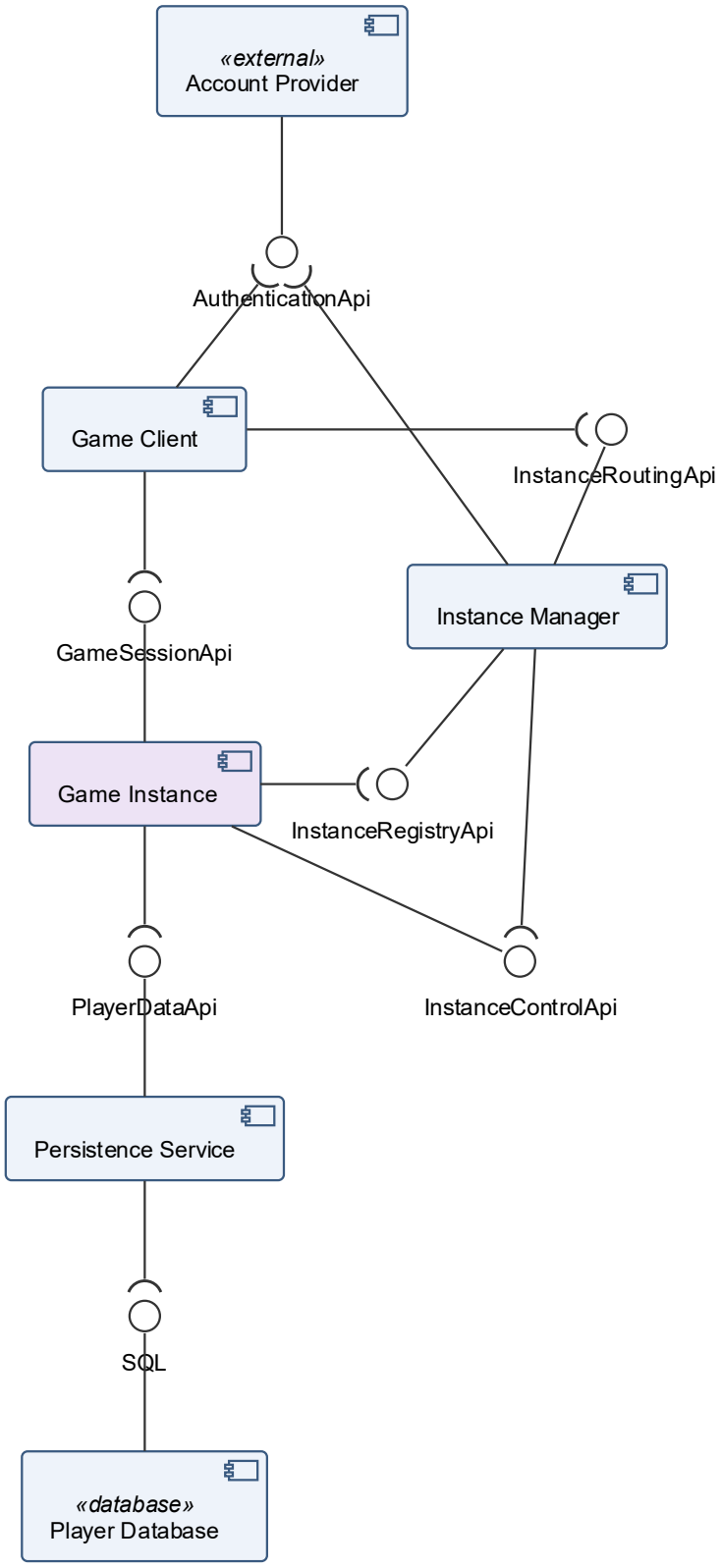


Figure 5.3: UML component diagram (layer 2)

Interface	Provided by	Required by	Main operations
AuthenticationApi	Account Provider	Game Client, Instance Manager	Login (OAuth 2.0), validateToken(token)
InstanceRoutingApi	Instance Manager	Game Client	requestOwnWorld(token) : InstanceAddress, requestVisit(token, ownerPlayerId) : InstanceAddress

Interface	Provided by	Required by	Main operations
InstanceRegistryApi	Instance Manager	Game Instance	register(instanceId, address), reportStatus(instanceId, playerCount)
InstanceControlApi	Game Instance	Instance Manager	shutdown()
GameSessionApi	Game Instance	Game Client	join(token), sendCommand(command); the instance pushes WorldUpdate and BattleUpdate messages
PlayerDataApi	Persistence Service	Game Instance	getPlayer(playerId), saveCharacter(...), saveInventory(...), savePosition(...)
SQL	Player Database	Persistence Service	Standard SQL over TCP

5.4 Instance life cycle

The instance rules of the case (R-C1 to R-C6) are handled by the Instance Manager together with the Game Instances. Figure 5.4 shows the most complex case: a player visits the world of another player.

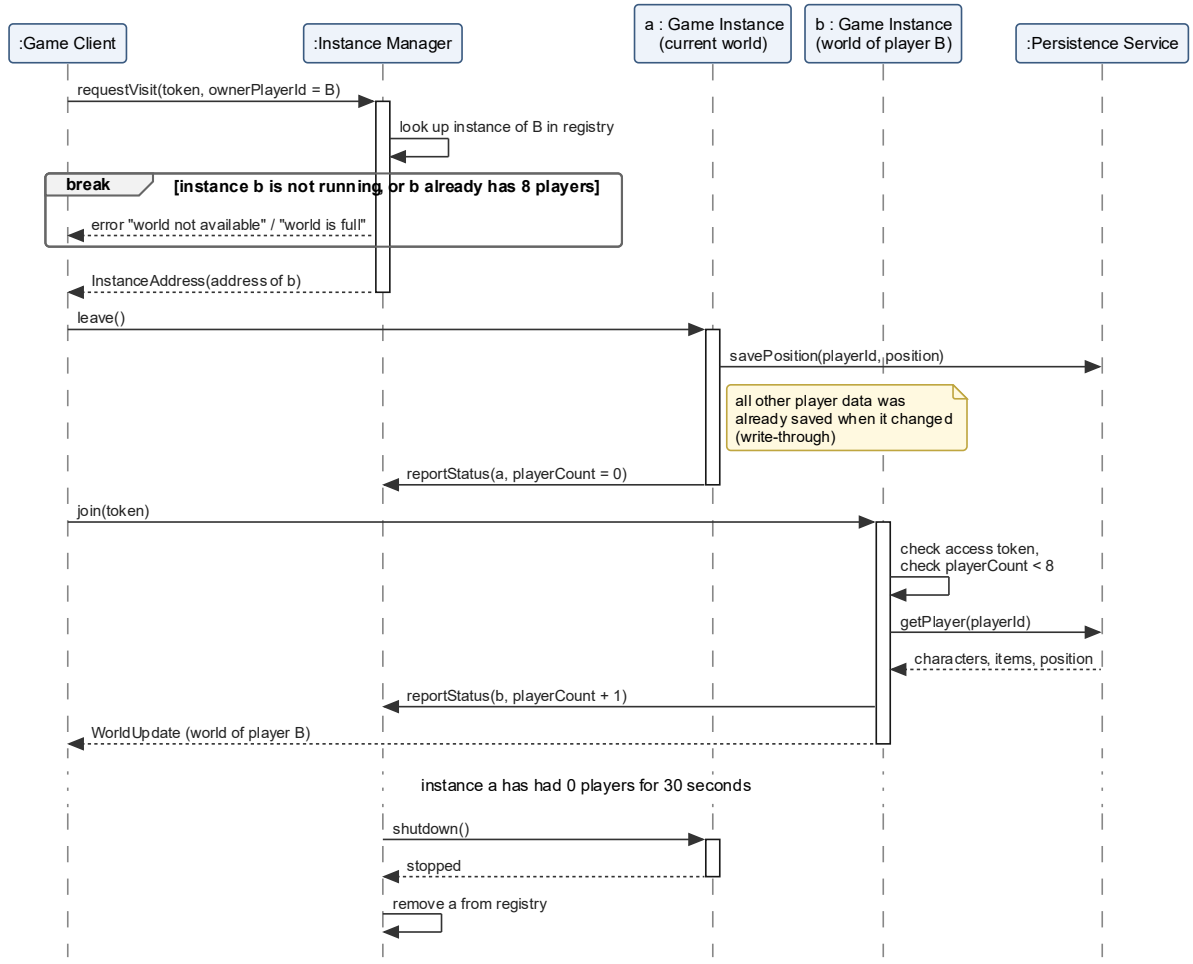


Figure 5.4: UML sequence diagram: visiting another player's world

1. **Own world (R-C2).** requestOwnWorld starts a new Game Instance container (or returns the running one if this player's world is already running), waits until it has registered and returns its address.
2. **Visit (R-C3, R-C5).** requestVisit checks the registry: the owner's instance must be running and have fewer than 8 players. The instance checks the limit again when the player actually joins, because two visitors can be sent to the same instance at the same moment.

3. **Shutdown (R-C4).** Every join and leave is reported to the Instance Manager. When an instance has had 0 players for 30 seconds, the Instance Manager stops it. The short delay prevents a restart when a player reconnects after a network problem.
4. **No data on the instance (R-C6).** Player data is saved the moment it changes (write-through, section 6.5). When the player leaves, only the position still has to be saved, so stopping the instance loses nothing.

5.5 Design notes

- **DN-2.1 Instance Manager as a separate container.** An instance cannot start itself and does not know about other instances. Routing and the limit of 8 players need one place with an overview of all instances. Alternative: clients connect directly to a known instance address. Rejected, because then nobody can enforce the limit or shut down empty instances.
- **DN-2.2 Persistence as a separate service.** Players constantly switch instances. If levels, weapons or the position were stored on the instance, the data would be lost or out of date the moment a player moves to another instance; combining persistence with the instance would mean losing player data every time an instance shuts down. Alternative: every instance writes to the database directly. Rejected, because every instance would need database credentials and knowledge of the schema, and every schema change would affect all instances.
- **DN-2.3 Battle Engine inside the Game Instance.** A battle belongs to one player on one instance. A separate battle service would add a network call to every action and a second place where game state lives. The engine is a library (battle-engine.jar) inside the Game Instance process.
- **DN-2.4 Same names in all views.** The table shows which container runs on which node.

Container (C4)	Node (UML deployment)	Artifact
Game Client	Player PC	game-client.jar
Instance Manager	Instance Manager Server	instance-manager.jar
Game Instance	Game Server, Game Instance container	game-instance image (contains battle-engine.jar)
Persistence Service	Persistence Server	persistence-service.jar
Player Database	Database Server	player_db schema

6. C4 Layer 3: Component

6.1 Component diagram

Figure 6.1 zooms in on the Game Instance container. The Battle Engine (purple) is the component that chapter 7 describes at code level. The other components show how the Battle Engine is used: who starts a battle, who chooses actions, who passes the turn and how results reach the client and the Persistence Service.

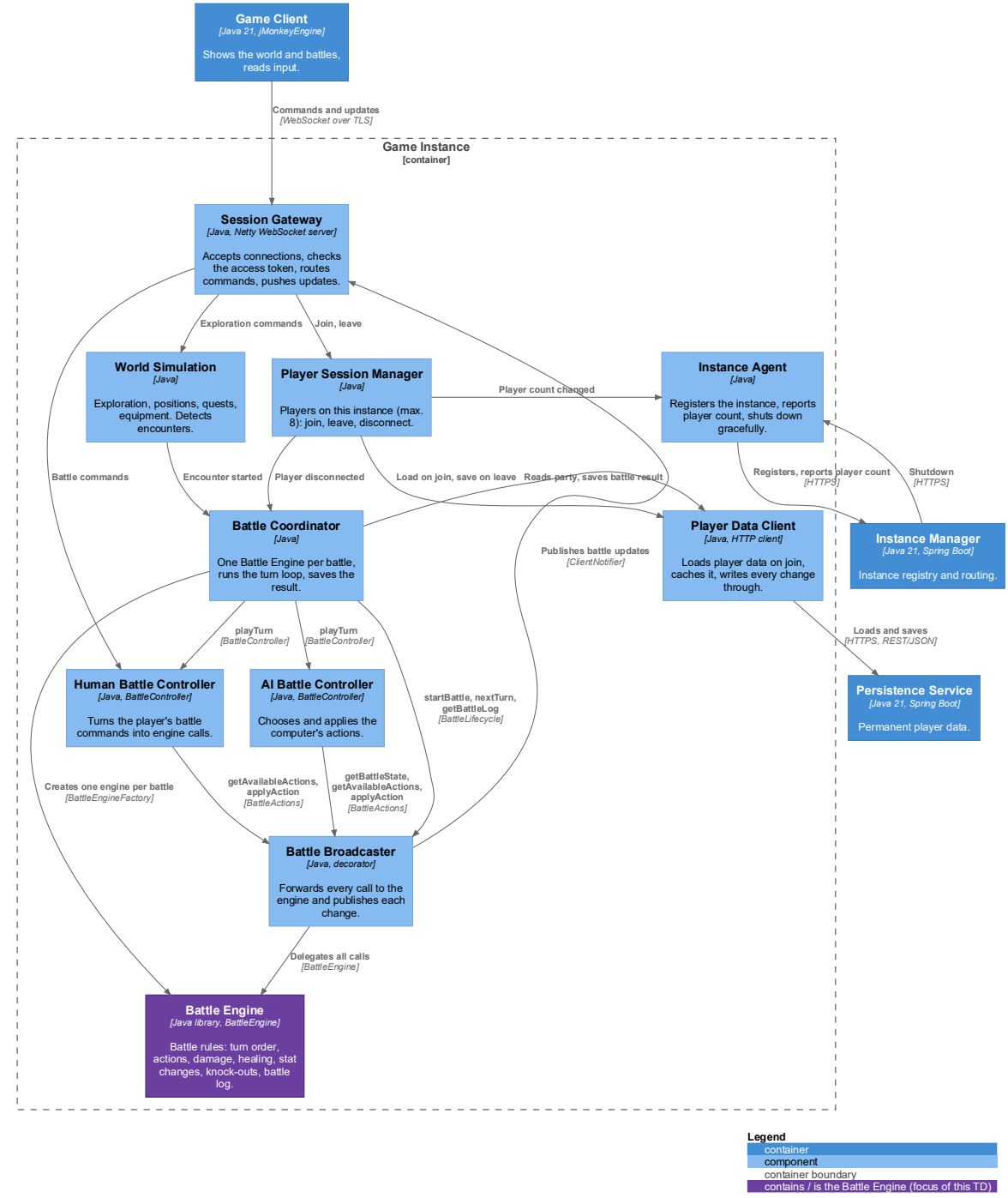


Figure 6.1: C4 component diagram of the Game Instance

Component	Responsibility	Provides
Session Gateway	Accepts WebSocket connections, checks the player's access token, sends every command to the right component and pushes updates to the clients.	GameSessionApi, ClientNotifier
Player Session Manager	Keeps the list of players on this instance (max. 8) and handles join, leave and disconnect.	PlayerSessions
Instance Agent	Registers the instance at the Instance Manager, reports the player count and shuts the process down gracefully.	InstanceStatus, InstanceControlApi
World Simulation	Exploration, positions, quests, navigation and equipment; detects enemy encounters.	WorldCommands
Battle Coordinator	Turns an encounter into a battle: builds ConfigBattle, creates one Battle Engine per battle, runs the turn loop, calls nextTurn and saves the result.	EncounterHandler
Human Battle Controller	Handles the player's battle commands: asks for available actions, applies the chosen action, ends the turn on "End turn" or when the turn timer expires.	BattleController, BattleCommands
AI Battle Controller	Plays the computer's turn: reads the state, chooses an action per unit and applies it.	BattleController
Battle Broadcaster	Decorator around the Battle Engine: forwards every call and, after every change, sends the new state and log lines to the player's client.	BattleActions, BattleLifecycle
Battle Engine	Runs the rules of one battle: turn order, validation, damage, healing, stat changes, knock-outs, end of battle and the log.	BattleEngine, BattleEngineFactory
Player Data Client	Loads a player's data when they join, keeps it in memory while they are on this instance and writes every change straight through to the Persistence Service.	PlayerDataCache

6.2 Component diagram (UML)

Figure 6.2 shows the same components with their interfaces. It makes two design choices visible. First, the Battle Broadcaster provides BattleActions and BattleLifecycle, the two halves of BattleEngine. The controllers only require BattleActions, so they cannot pass the turn or start a battle; only the Battle Coordinator requires BattleLifecycle (Interface Segregation, section 8.1). Second, the Human and the AI Battle Controller provide the same interface, BattleController, so the Battle Coordinator treats both players the same way (Strategy, section 8.2).

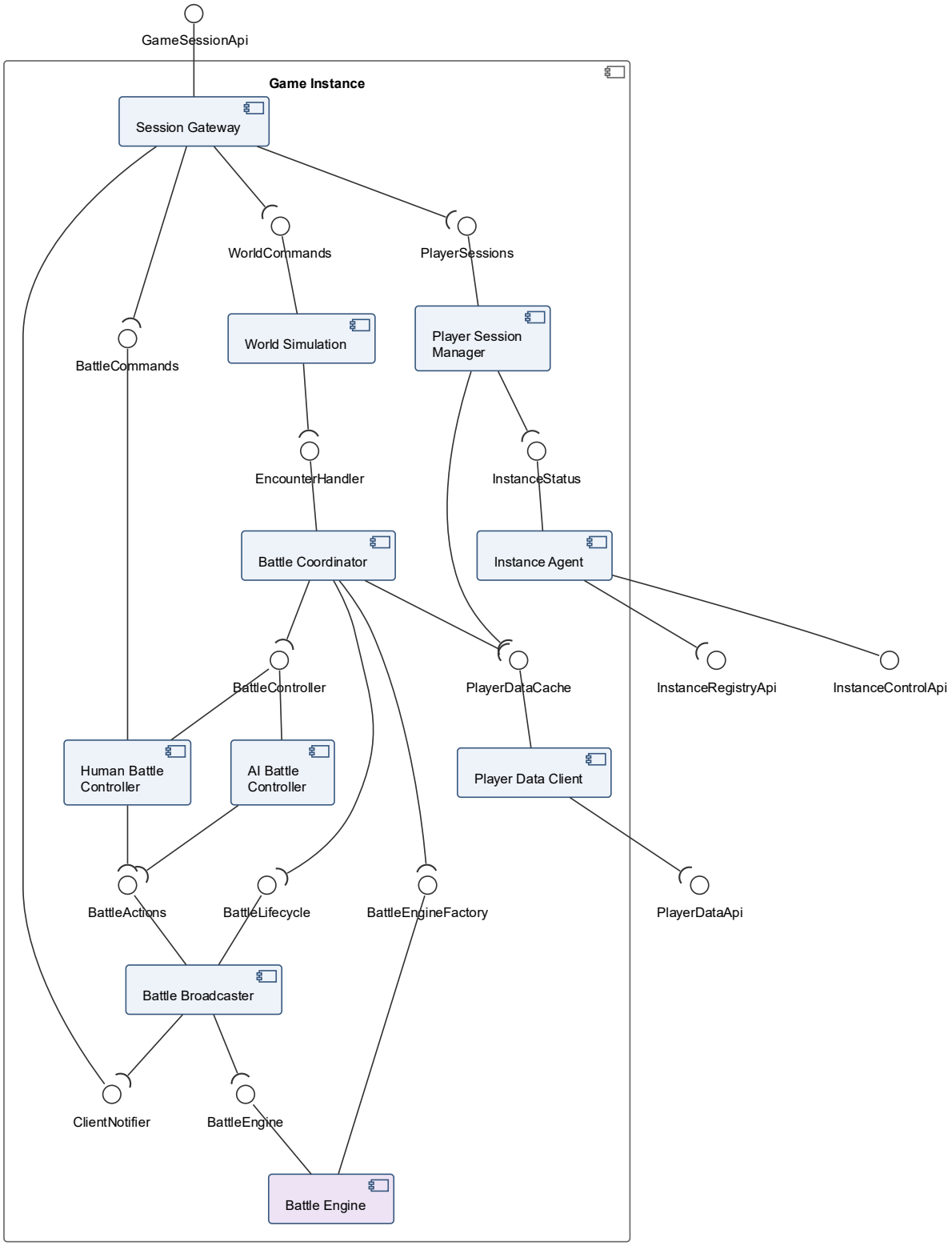


Figure 6.2: UML component diagram of the Game Instance (layer 3)

6.3 Deployment diagram (detailed)

Figure 6.3 zooms in on the Game Server node. A Game Instance container holds a Java 21 runtime with the jar files of the instance and a deployment specification with the settings of that instance. The Battle Engine is a separate jar file, battle-engine.jar, which is only used by game-instance-battle.jar and itself depends on no other jar of the game.

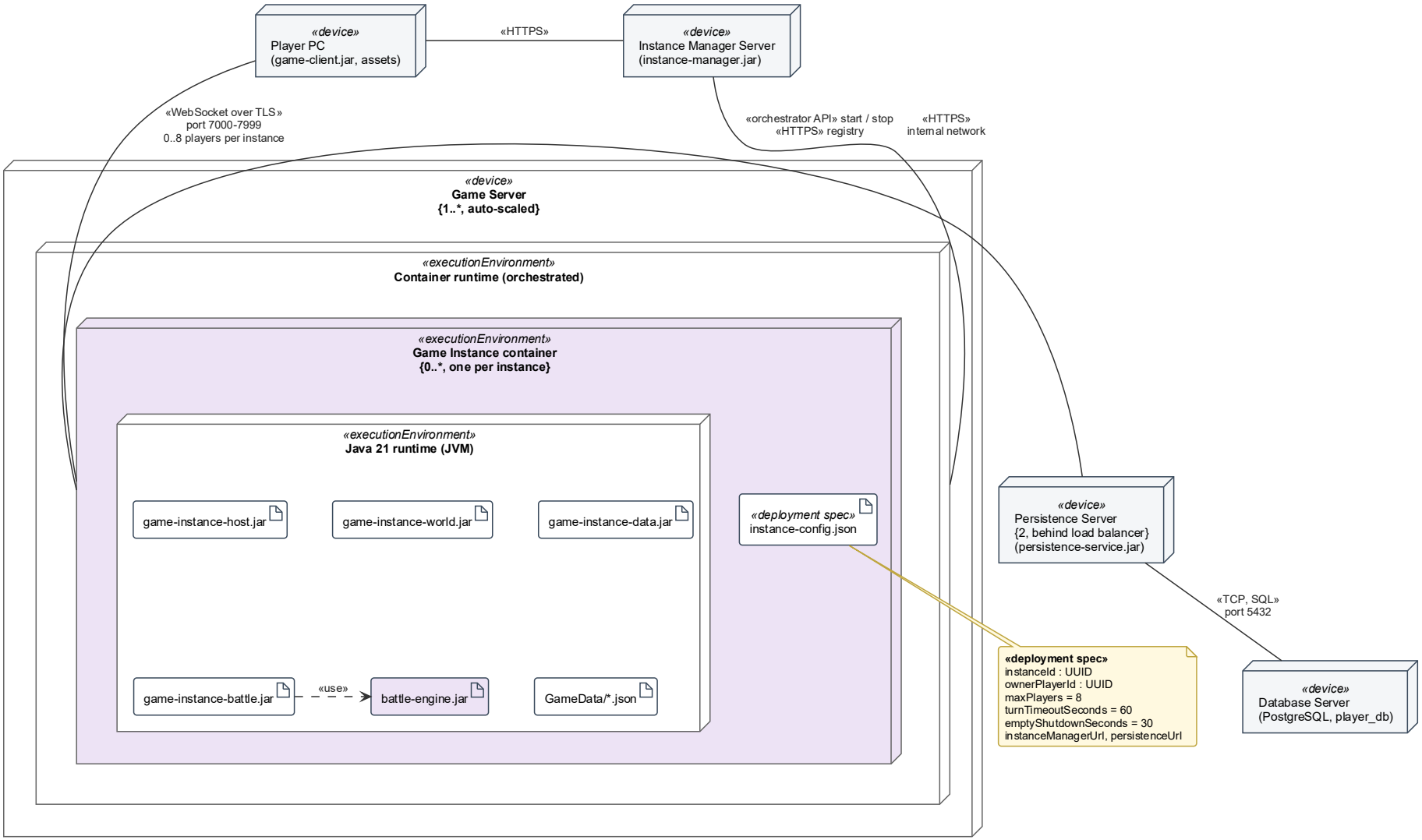


Figure 6.3: UML deployment diagram of the Game Server (layer 3)

Artifact	Contains (manifests)
game-instance-host.jar	Session Gateway, Player Session Manager, Instance Agent
game-instance-world.jar	World Simulation
game-instance-battle.jar	Battle Coordinator, Human Battle Controller, AI Battle Controller, Battle Broadcaster
game-instance-data.jar	Player Data Client
battle-engine.jar	Battle Engine
GameData/*.json	Game content: enemy groups, encounter tables, action definitions
instance-config.json	Deployment specification: instanceId, ownerPlayerId, maxPlayers = 8, turnTimeoutSeconds = 60, emptyShutdownSeconds = 30, addresses of the Instance Manager and Persistence Service

Because these settings are in a deployment specification, the limit of 8 players, the turn timer and the shutdown delay can be changed without building a new version.

6.4 Dynamic view

6.4.1 From encounter to battle

Figure 6.4 shows how a battle starts. The World Simulation detects an encounter and informs the Battle Coordinator. The Battle Coordinator gets the player's party from the Player Data Client (from memory, without a network call) and builds the ConfigBattle. This is where levels and equipment are translated into plain numbers: the Power, Defence and Spirit of a character include the bonuses of its equipment, the HP is the saved HP and the actions are the actions the character knows at its level. The enemies come from the encounter tables in GameData. The Battle Engine only receives these numbers and knows nothing about levels or equipment.

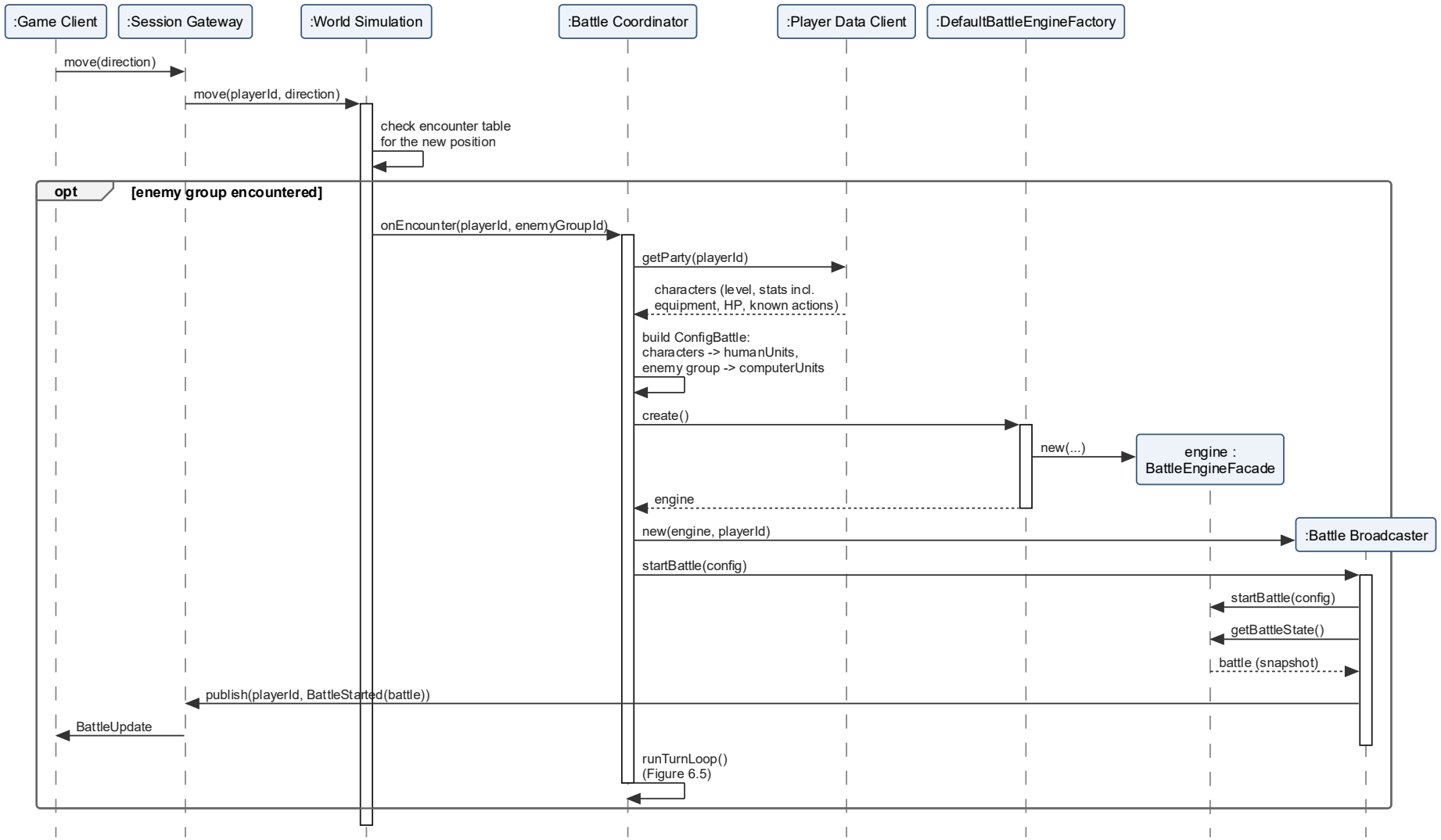


Figure 6.4: UML sequence diagram: an encounter starts a battle

6.4.2 Playing turns

Figure 6.5 shows the turn loop of the Battle Coordinator. It asks which player is active and lets the matching controller play the turn. The human turn ends when the player chooses "End turn" or when the turn timer of 60 seconds expires, so a player who walks away cannot block the game. After each turn the Battle Coordinator calls nextTurn(). When the battle is finished, it saves the result (experience, level-ups, remaining HP) through the Player Data Client.

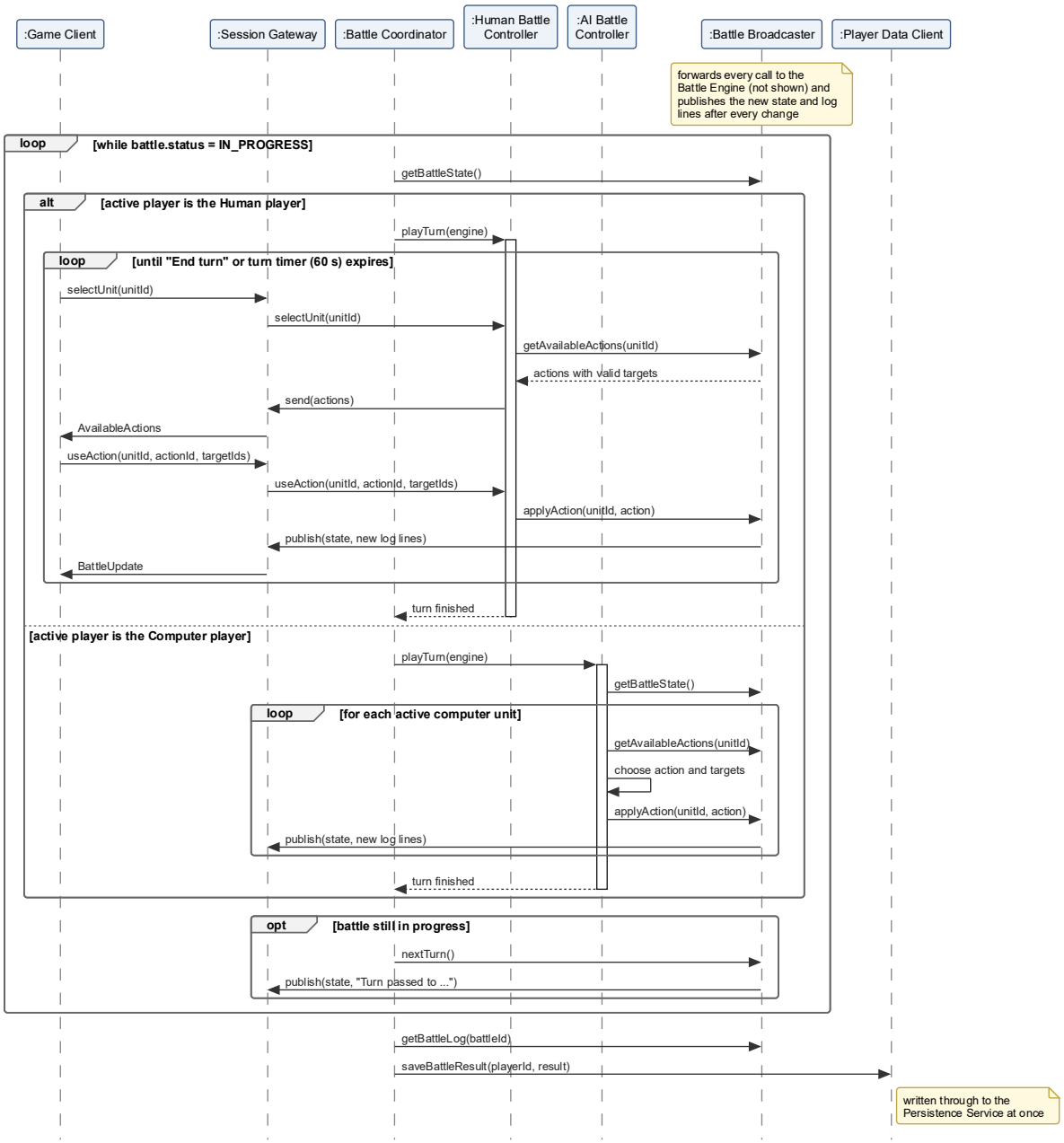


Figure 6.5: UML sequence diagram: the turn loop

6.5 Design notes

- **DN-3.1 Only one component calls nextTurn().** The Battle Coordinator owns the turn loop and is the only component that calls startBattle and nextTurn. The controllers only get BattleActions, which makes it impossible for the AI to pass the human player's turn by mistake.

- **DN-3.2 BattleController with two implementations.** The Battle Coordinator does not need to know whether a human or the computer plays a turn. New kinds of controllers, for example an auto-battle option in which the AI plays for the player, can be added without changing the Battle Coordinator.
- **DN-3.3 Battle Broadcaster as a decorator.** The engine stays free of networking (Single Responsibility). Without the decorator, both controllers and the Battle Coordinator would have to send updates themselves, so the same code would exist three times.
- **DN-3.4 Write-through cache in the Player Data Client.** Reading from memory during a session is fast, and writing every change immediately means nothing is lost when an instance stops or crashes (R-C6). Alternative: save only when the player leaves. Rejected, because after a crash the progress would be gone and the next instance would load old data.
- **DN-3.5 The Battle Coordinator decides what is saved.** After a battle it saves experience, level-ups and the remaining HP. Stat changes from the battle are temporary and are not saved.
- **DN-3.6 Disconnect during a battle.** The Player Session Manager informs the Battle Coordinator, which abandons the battle (I-12).
- **DN-3.7 Change since version 0.1.** In version 0.1 the Battle Engine called a persistence client itself. That contradicted the goal (the engine does not know the rest of the game). Now the Battle Coordinator saves the results and the engine has no dependencies.

7. C4 Layer 4: Code

This chapter zooms in on the Battle Engine component. It contains the data model and all classes that are needed to implement the engine.

7.1 Package overview

The engine is one jar file, battle-engine.jar, with six Java packages. Figure 7.1 shows them with their dependencies. All dependencies point downwards: there are no cycles, and the Model and Logging packages do not depend on the packages that use them.

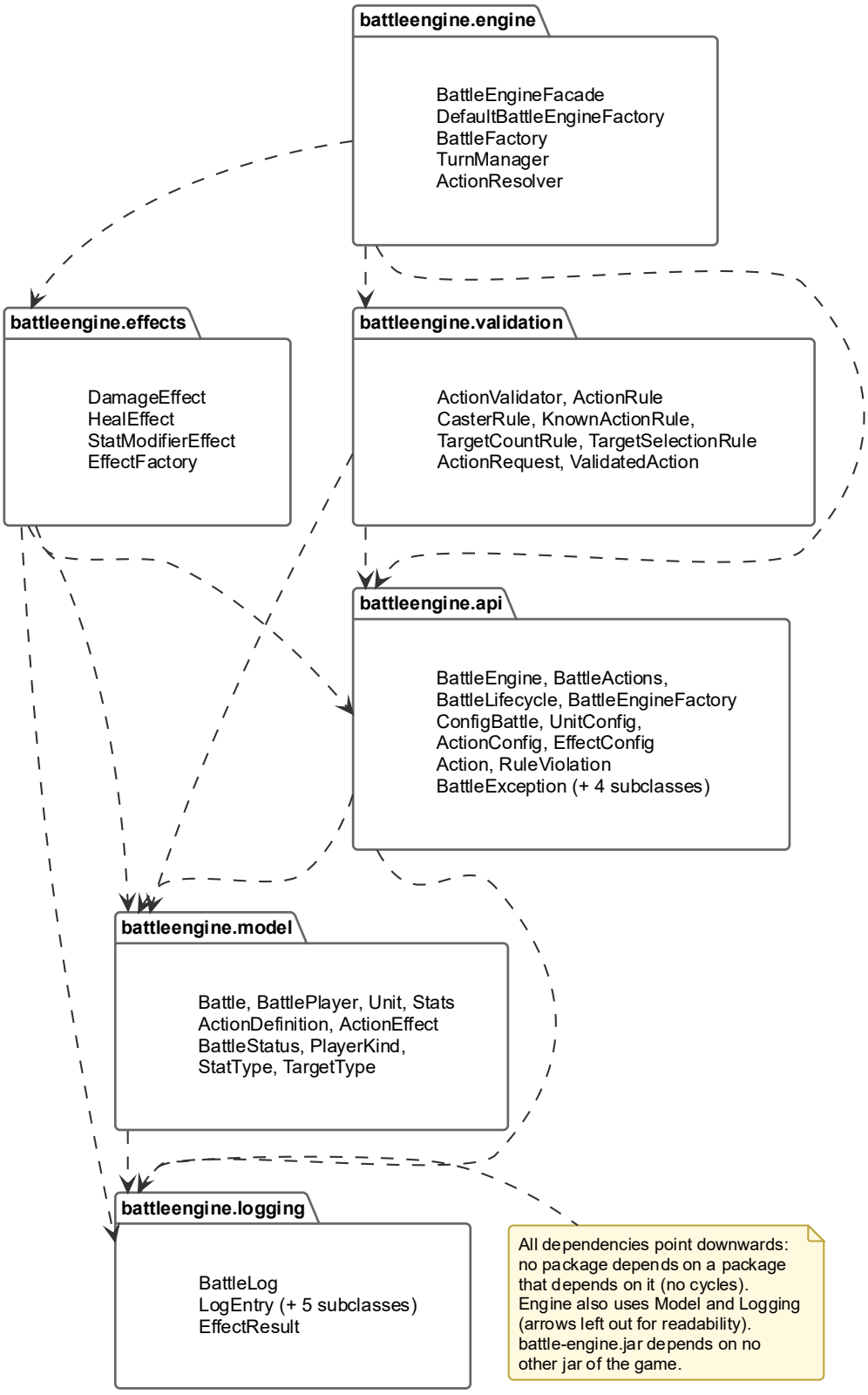


Figure 7.1: UML package diagram of the Battle Engine

Package	Contents	Responsibility
battleengine.api	Interfaces, ConfigBattle, Action, exceptions	Everything other components use.
battleengine.engine	BattleEngineFacade (facade), factories, TurnManager, ActionResolver	Coordinates a battle.
battleengine.validation	ActionValidator and the rules	Decides whether an action is allowed.
battleengine.effects	The effect strategies and EffectFactory	Damage, healing and stat formulas.
battleengine.model	Battle, BattlePlayer, Unit, Stats, ActionDefinition	The state of a battle and its invariants.
battleengine.logging	BattleLog, log entries, EffectResult	What happened in the battle.

7.2 Interface and facade

Figure 7.2 shows the interface and the facade. BattleEngine contains exactly the six methods of the assignment, split into two smaller interfaces: BattleActions for the controllers and BattleLifecycle for the Battle Coordinator (ISP). BattleEngineFacade is the only class that implements it. It is a facade that hides the BattleFactory, the TurnManager, the ActionValidator and the ActionResolver. One BattleEngineFacade object runs one battle; the DefaultBattleEngineFactory creates a new, fully wired engine for every battle.

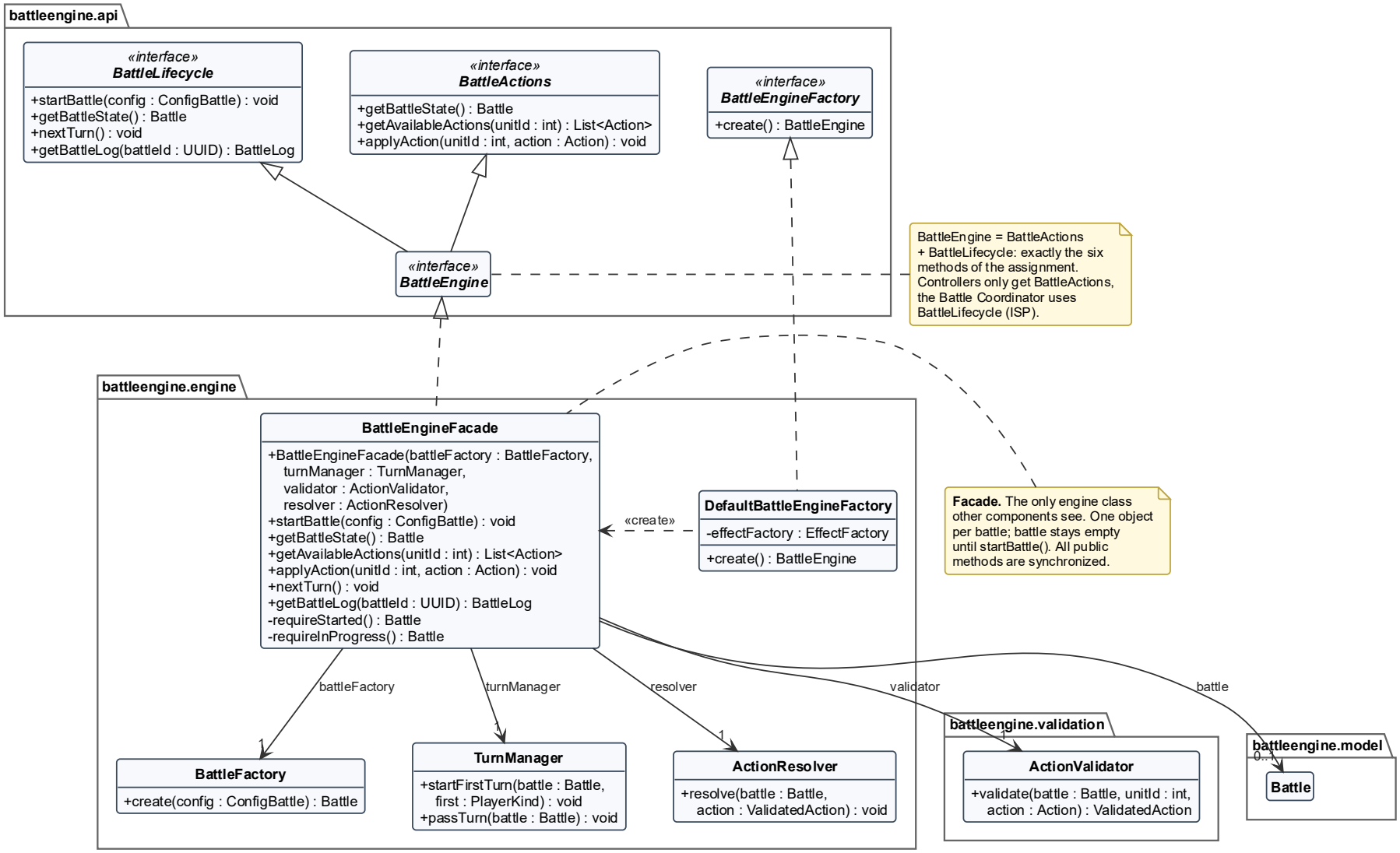


Figure 7.2: UML class diagram: interface and facade

Method	Allowed when	Behaviour	Errors
startBattle(config)	not started	BattleFactory validates the config and builds the Battle; TurnManager makes the first player active. Log: "Battle starts", "Turn passed to player 1 (Human)".	InvalidBattleConfigException; BattleStateException if already started
getBattleState()	started	Returns `battle.createSnapshot()`, a deep copy.	BattleStateException if not started
getAvailableActions(unitId)	started	Empty list if the battle is finished or the unit is knocked out or not on the active side. Otherwise one Action per ActionDefinition of the unit that has at least one valid target, with validTargetIds filled in.	InvalidActionException (UNKNOWN_UNIT)
applyAction(unitId, action)	in progress	`ActionValidator.validate()`, then `ActionResolver.resolve()` (section 7.9).	InvalidActionException; BattleStateException if not in progress
nextTurn()	in progress	`TurnManager.passTurn()`: the opponent of the active player becomes active, turnNumber + 1, log "Turn passed to ...".	BattleStateException if not in progress
getBattleLog(battleId)	started	Returns a copy of the log if battleId is the id of this battle.	UnknownBattleException

All errors are subclasses of BattleException (Figure 7.3), so a caller can catch one type. Every public method of BattleEngineFacade is `synchronized`: messages from the network arrive on different threads, and this guarantees that two calls on the same battle never run at the same time. Different battles have different engine objects and do not block each other.

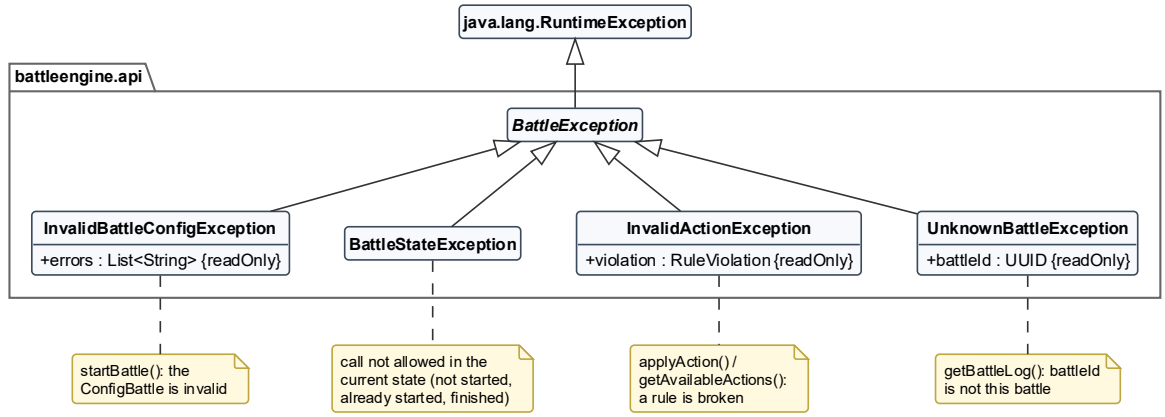


Figure 7.3: UML class diagram: exceptions

7.3 Battle model

Figure 7.4 shows the state of a battle. Battle is the aggregate: it owns exactly two BattlePlayers (R-T2) and points to exactly one of them as the active player (R-T1). I named the class BattlePlayer instead of Player, because "Player" already means the person in layers 1 to 3. A BattlePlayer owns its units in two lists: active units and knocked-out units. When a unit is knocked out it moves to the second list (I-9), so it can no longer act or be targeted, but the state is still complete.

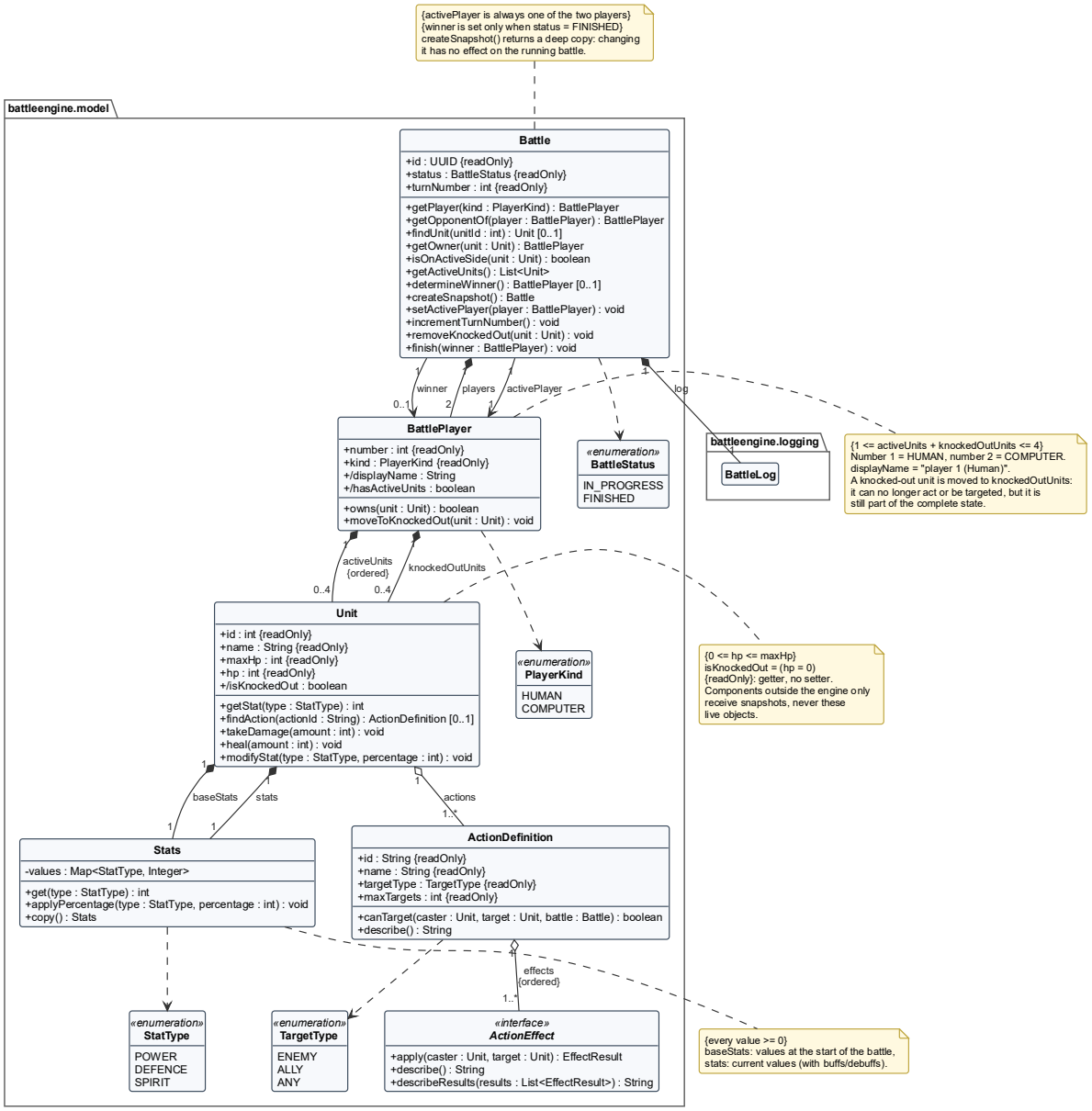


Figure 7.4: UML class diagram: battle model

- **Unit** guards the invariant 0 ≤ hp ≤ maxHp itself: `takeDamage()` never goes below 0 and `heal()` never above maxHp (R-U1, R-U3). isKnockedOut is derived: hp = 0.
- **Stats** stores the values per StatType in a map, so adding a stat (for example Speed) does not change the class. A unit has baseStats (the values at the start) and stats (the current values, including buffs and debuffs).
- **No live objects outside the engine.** The methods that change the model are public, because the effects, the TurnManager and the ActionResolver are in other packages. Still, no component outside the engine can change a running battle: the BattleEngineFacade never hands out the live Battle object, only snapshots. Every change therefore goes through the rules of applyAction and nextTurn.
- **Snapshot.** getBattleState() returns `battle.createSnapshot()`: a deep copy of the battle with its players, units and log. The caller gets a consistent picture that does not change while it reads it, and changing the copy has no effect on the battle. With at most 8 units and a few hundred log entries the copy is cheap.

7.4 Actions and effects

Figure 7.5 shows how actions work. There are two classes for two different things. **ActionDefinition** is what a unit can do: a name, a TargetType, a maximum number of targets and an ordered list of 1..* effects. **Action** is the object of the interface: a request to use one ActionDefinition (actionId) by one caster (casterId) on chosen targets (targetIds). getAvailableActions() returns Actions with the targets the unit may choose (validTargetIds); the caller fills in targetIds and passes the Action to applyAction(). This solves the problem that an Action must contain the targets (R-A2), while the targets are not known yet when the available actions are requested.

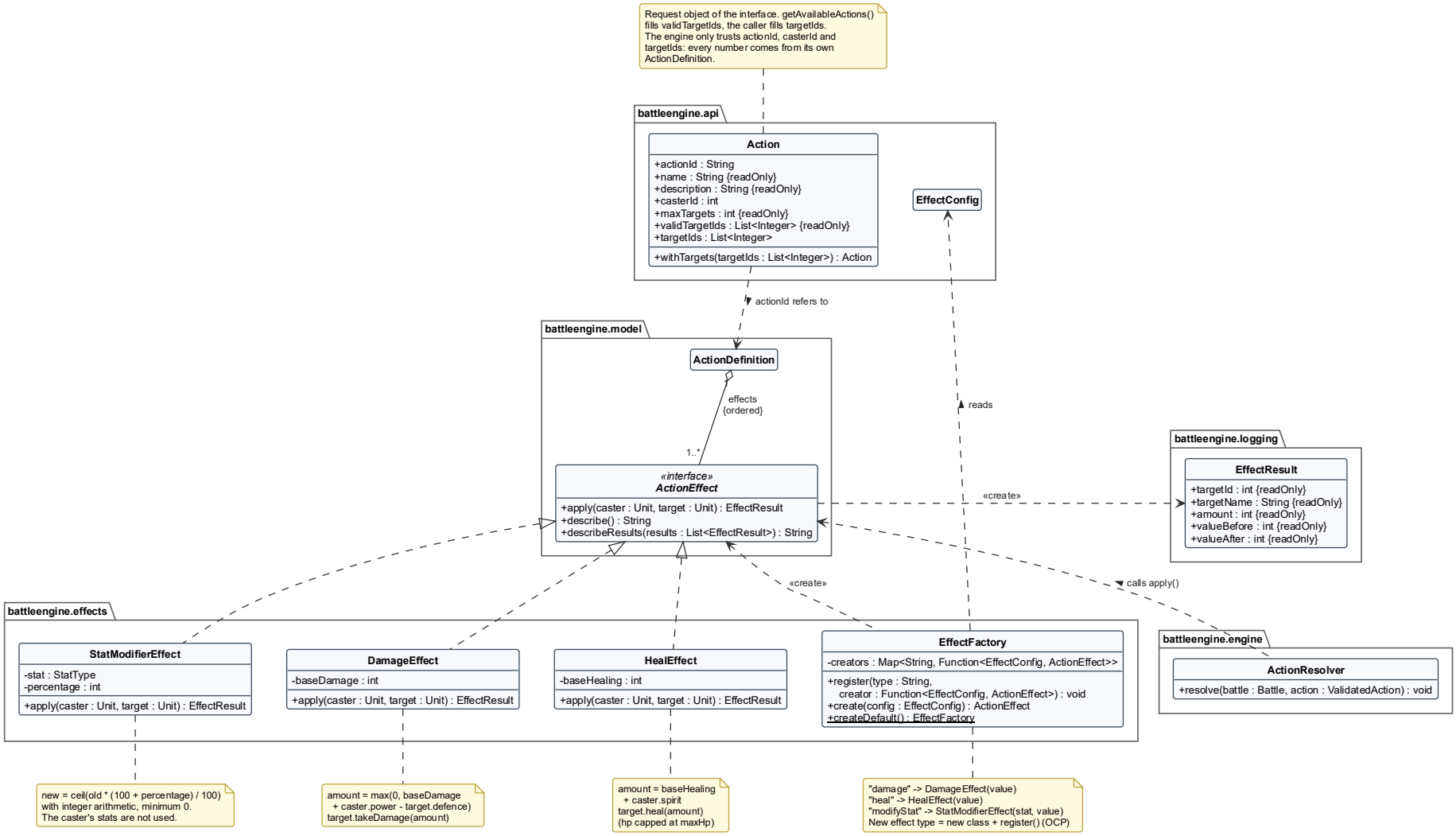


Figure 7.5: UML class diagram: actions and effects

The effects use the Strategy pattern: ActionEffect has one implementation per action type of the assignment (R-A3 to R-A5), and ActionResolver calls `apply(caster, target)` without knowing which effect it is. Because an ActionDefinition has a list of effects, an action like "hit and lower Defence" is a combination of two existing effects and needs no new code. The EffectFactory creates the effects from configuration: it maps a type name ("damage", "heal", "modifyStat") to a constructor. A new effect type needs one new class and one `register()` call.

Server authority. The engine only uses actionId, casterId and targetIds of the Action it receives. Damage, healing and percentages always come from its own ActionDefinition, so a modified client that sends an Action with 9999 damage has no effect.

7.5 Validation

Figure 7.6 shows how applyAction() checks a request. The ActionValidator looks up the caster, the ActionDefinition and the targets, and then runs a list of rules in a fixed order. The first rule that finds a problem stops the check: an InvalidActionException with a RuleViolation is thrown and nothing in the battle has changed. Only after all rules pass does the ActionResolver change the battle.

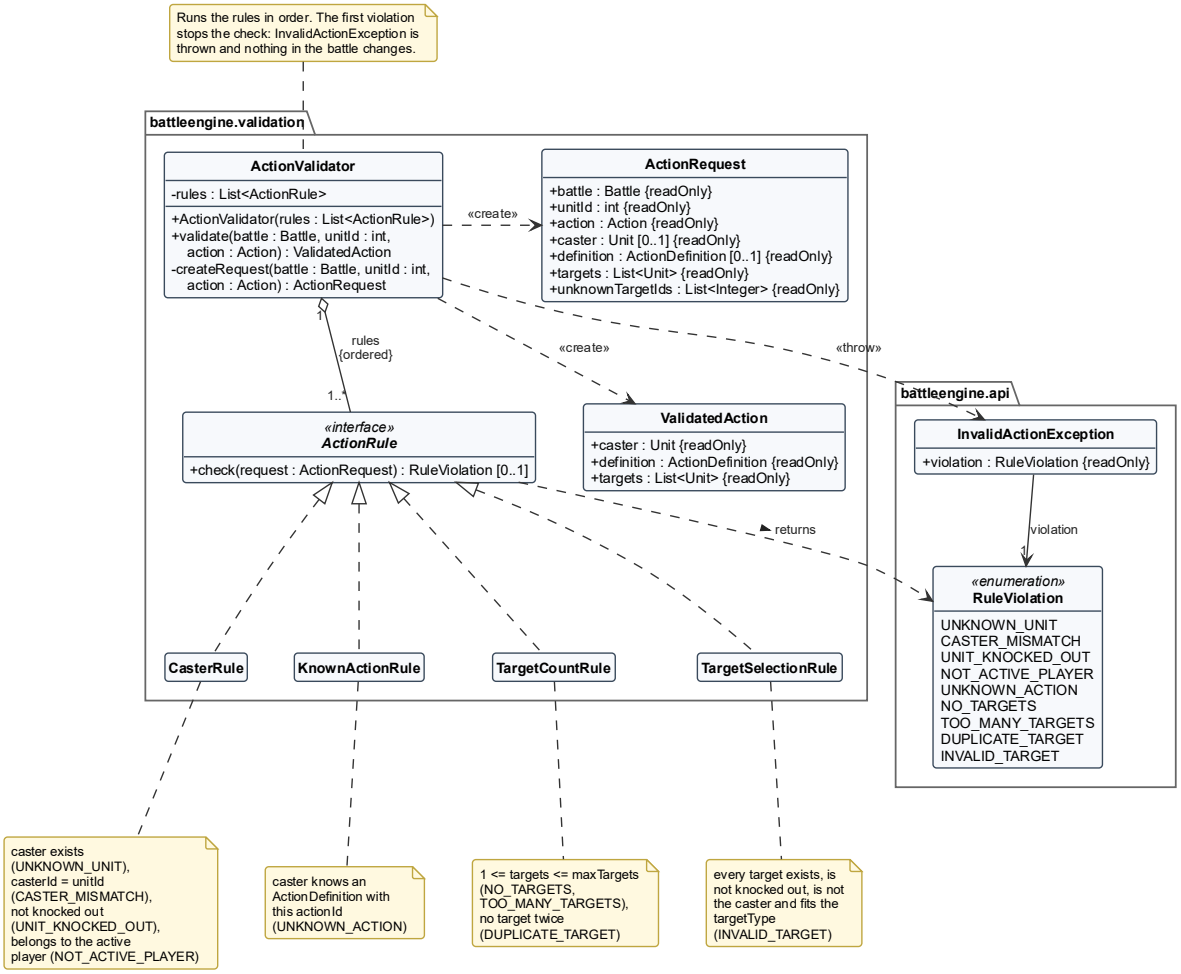


Figure 7.6: UML class diagram: validation rules

Rule	Checks	Violation	Requirement
CasterRule	The unit exists, action.casterId = unitId, the unit is not knocked out and belongs to the active player.	UNKNOWN_UNIT, CASTER_MISMATCH, UNIT_KNOCKED_OUT, NOT_ACTIVE_PLAYER	R-T5, R-A1, R-U2
KnownActionRule	The caster has an ActionDefinition with this actionId.	UNKNOWN_ACTION	R-A2
TargetCountRule	1 ≤ number of targets ≤ maxTargets, and no target twice.	NO_TARGETS, TOO_MANY_TARGETS, DUPLICATE_TARGET	R-A2, I-5
TargetSelectionRule	Every target exists, is not knocked out, is not the caster and fits the TargetType of the action.	INVALID_TARGET	R-A2, R-U2, I-4, I-5

Because the rules are separate objects in a list, a new rule (for example "every unit may act once per turn") is a new class that is added to the list in the DefaultBattleEngineFactory; the ActionValidator itself does not change.

7.6 Configuration and factories

Figure 7.7 shows ConfigBattle, the parameter of startBattle(). It contains everything the engine needs: the battle id, the player that starts and 1-4 units per player (R-C8), each with HP, the three stats and its actions with their effects. The BattleFactory turns a ConfigBattle into a Battle. It first checks the whole configuration and collects all errors, so the caller sees every problem at once.

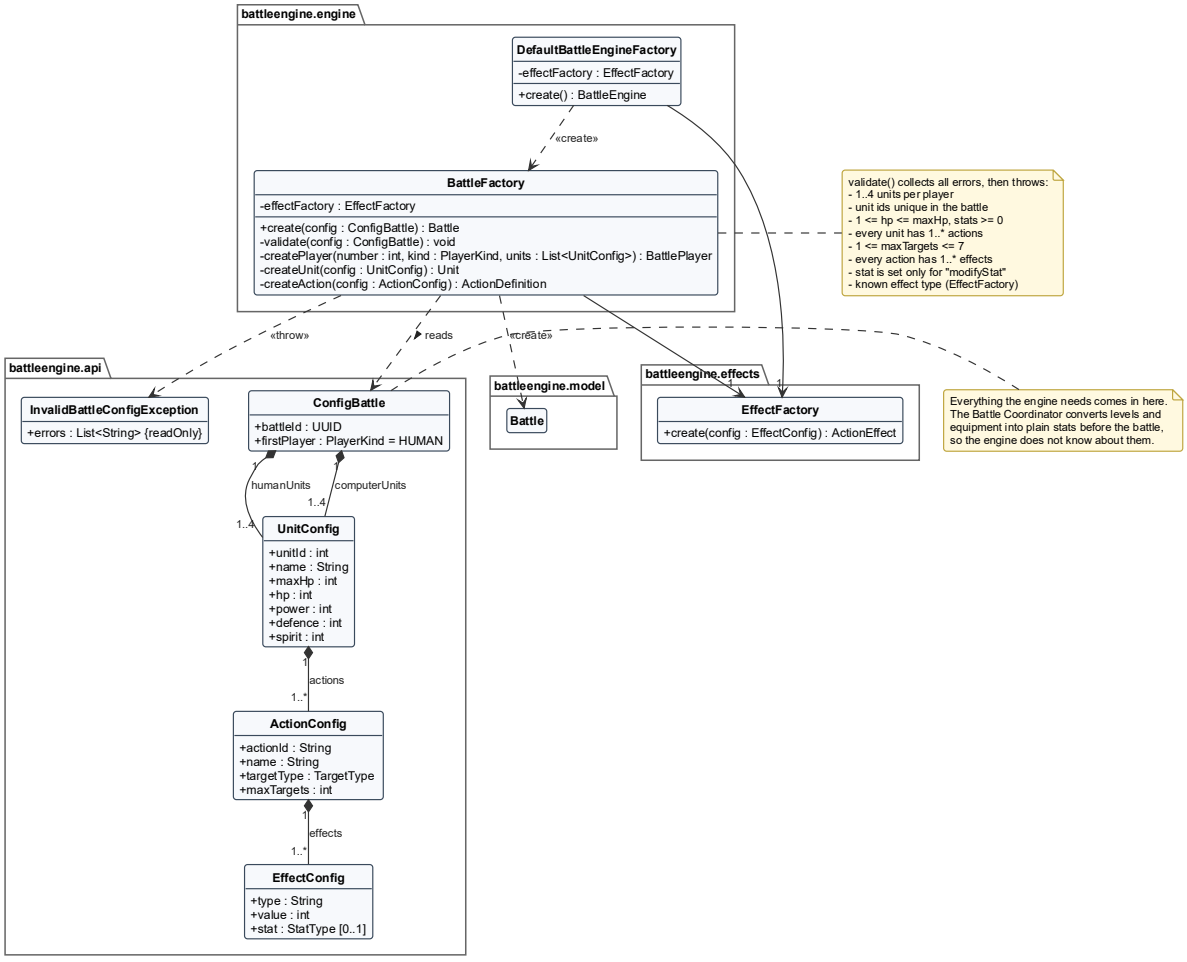


Figure 7.7: UML class diagram: configuration and factories

7.7 Battle log

Figure 7.8 shows the log. BattleLog is an ordered, append-only list of LogEntry objects. Every kind of event has its own subclass that describes itself in one line (`toText()`). An ActionPerformedEntry also keeps the EffectResults: per target the calculated amount and the value before and after. The client can use them for animations and damage numbers.

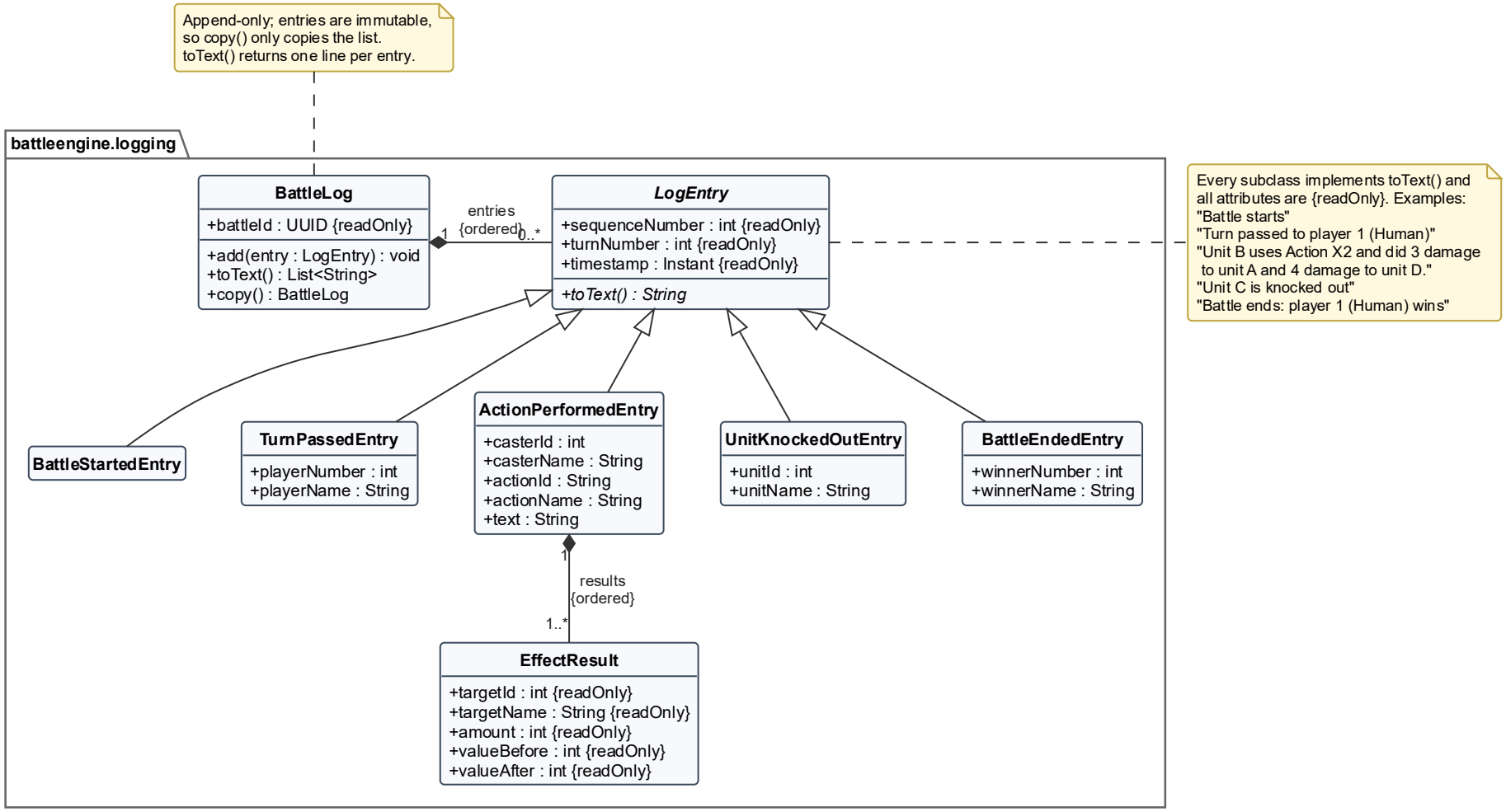


Figure 7.8: UML class diagram: battle log

The log of an example battle, as returned by `getBattleLog(battleId).toText()`, follows the example of the assignment:

```
Battle starts
Turn passed to player 1 (Human)
Unit A uses Action X1 and did 15 damage to unit B.
Unit A uses Action X1 and did 12 damage to unit C.
Unit D uses Action X2 and heals unit A for 12.
Turn passed to player 2 (Computer)
Unit B uses Action X2 and did 3 damage to unit A and 4 damage to unit D.
Turn passed to player 1 (Human)
...
Unit C is knocked out
Battle ends: player 1 (Human) wins
```

The text of an ActionPerformedEntry is built by the effects themselves (`ActionEffect.describeResults()`): a damage effect writes "did 3 damage to unit A and 4 damage to unit D", a heal effect "heals unit A for 12" and a stat effect "raises Power of unit A from 8 to 10". A new effect type therefore brings its own log text.

7.8 Life cycle of a battle

Figure 7.9 shows the states of a BattleEngineFacade object. It makes R-T1 visible: as soon as the battle is in progress, exactly one of the states HumanTurn and ComputerTurn is active, and only nextTurn() switches between them. A call that is not allowed in a state ends in an exception, so a caller can never bring the engine into an invalid state.

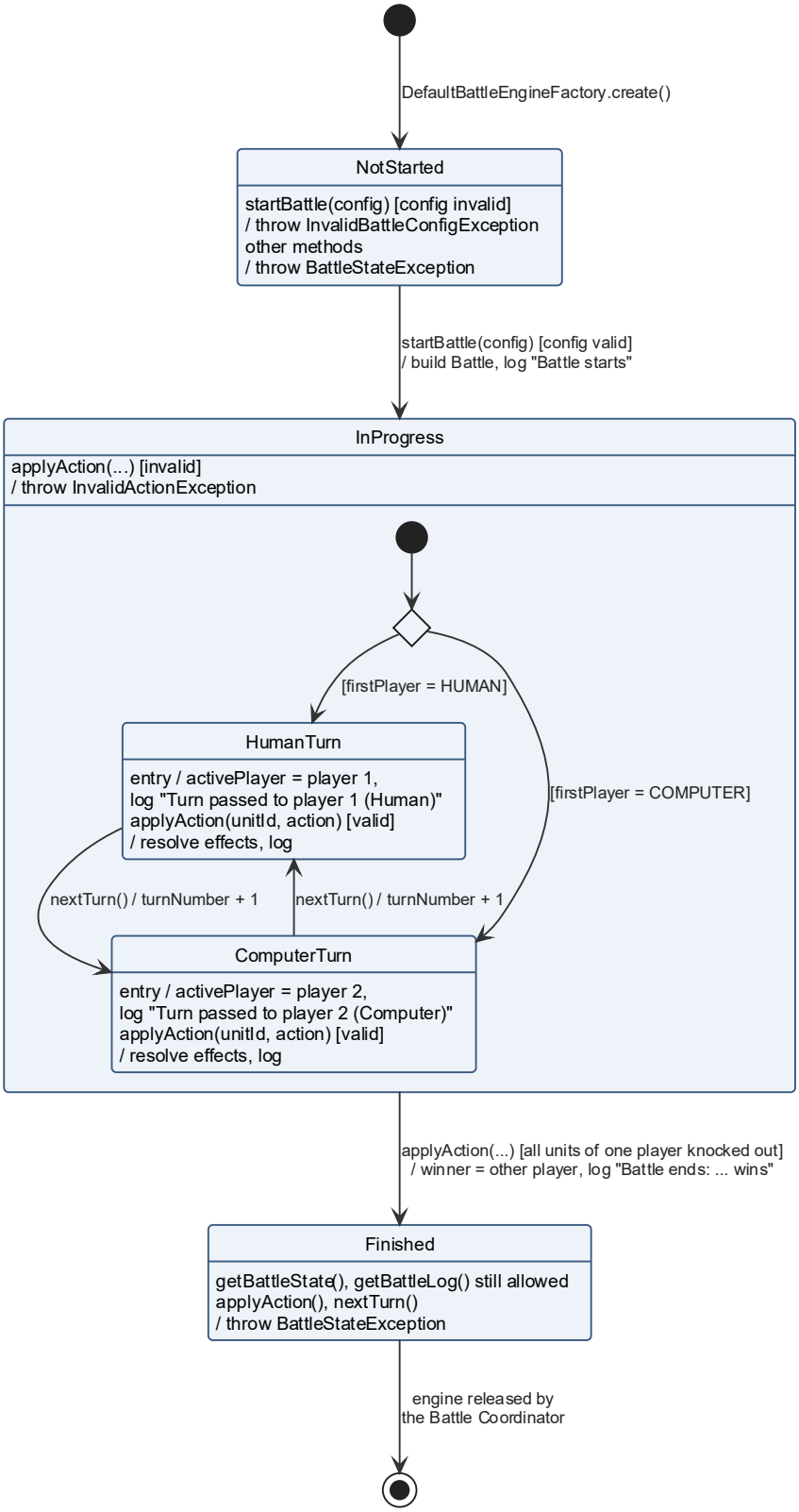


Figure 7.9: UML state machine diagram of a BattleEngineFacade

7.9 applyAction step by step

Figure 7.10 shows what happens inside the engine during applyAction():

1. BattleEngineFacade checks that the battle is in progress.

2. ActionValidator builds an ActionRequest and runs the rules (section 7.5). On a violation it throws an InvalidActionException and the battle is unchanged.
3. ActionResolver applies every effect of the ActionDefinition, in order, to every target that is not knocked out. It applies one effect to all targets before the next effect, so the log reads "did 3 damage to unit A and 4 damage to unit D".
4. It adds one ActionPerformedEntry with all EffectResults to the log.
5. Every target that now has 0 HP is moved to the knocked-out units, with a UnitKnockedOutEntry per unit (R-U2).
6. If one player has no active units left, the battle is finished, the other player wins and a BattleEndedEntry is added (I-2).

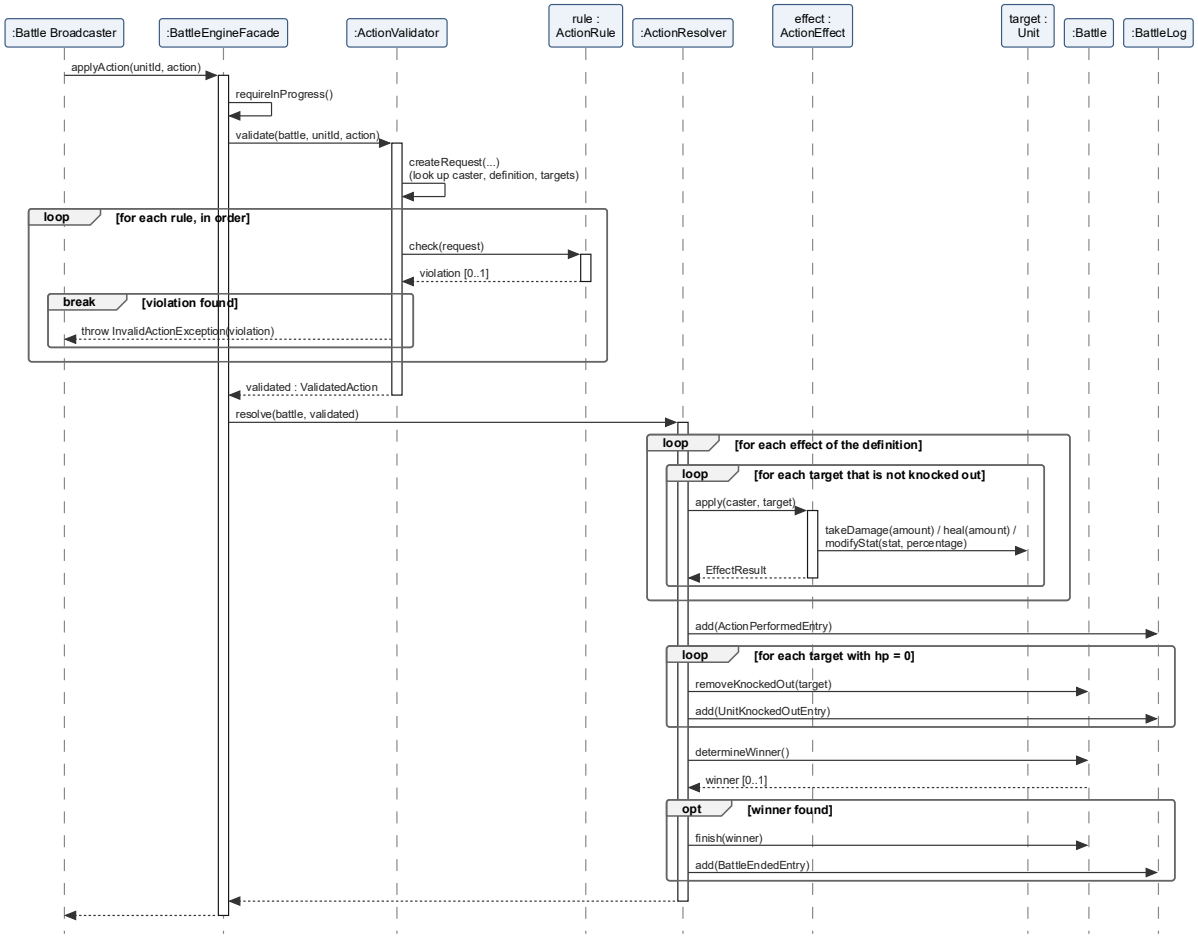


Figure 7.10: UML sequence diagram: applyAction inside the engine

7.10 Implementation of the rules

Some requirements are calculations rather than structure. This section shows where they are implemented and checks them against the examples of the assignment.

Rule	Implemented in	Formula or behaviour
Damage (R-U4)	DamageEffect.apply()	amount = max(0, baseDamage + caster.power - target.defence); target.takeDamage(amount)
Healing (R-U5)	HealEffect.apply()	amount = baseHealing + caster.spirit; target.heal(amount)
Max HP (R-U3)	Unit.heal()	hp = min(maxHp, hp + amount)
Knock-out (R-U2)	Unit.takeDamage(), ActionResolver	hp = max(0, hp - amount); a unit with hp = 0 is moved to knockedOutUnits
Stat change (R-A5, R-A6)	StatModifierEffect.apply(), Stats.applyPercentage()	new = ceil(old × (100 + p) / 100), minimum 0; the caster is not used
Turn order (R-T1, R-T4)	TurnManager.passTurn()	active = battle.getOpponentOf(active); turnNumber + 1

Rounding with integers. The rounding up of stat changes is calculated with integers: new = (old × (100 + p) + 99) div 100. With floating point numbers it goes wrong: in a computer 10 × 1.1 is 11.000000000000002, and rounding that up gives 12 instead of 11. With integers: (10 × 110 + 99) div 100 = 1199 div 100 = 11. For p ≤ -100 the result is 0.

Check with the examples of the assignment.

Example	Calculation in the design	Result
Damage 10, caster Power 4, target A Defence 6	max(0, 10 + 4 - 6)	8, as expected
Damage 10, caster Power 4, target B Defence 1	max(0, 10 + 4 - 1)	13, as expected
Heal 15, caster Spirit 6, targets A, B and C	15 + 6 for every target	21 each, as expected
Unit with 40 of 50 HP healed for 20	min(50, 40 + 20)	50, as expected
Power +25%, target A with Power 8	(8 × 125 + 99) div 100 = 1099 div 100	10, as expected
Power +25%, target B with Power 1	(1 × 125 + 99) div 100 = 224 div 100	2, as expected
Defence -25%, target with Defence 7 (own example)	(7 × 75 + 99) div 100 = 624 div 100	6 (5.25 rounded up)
13 damage on a unit with 5 HP (own example)	max(0, 5 - 13)	0 HP, knocked out

8. SOLID principles and design patterns

8.1 SOLID principles

Principle	Where in the design	Why
Single Responsibility	Every engine class has one reason to change: BattleEngineFacade coordinates, TurnManager owns the turn order, ActionValidator and the rules decide what is allowed, ActionResolver executes, the effects calculate, Unit guards its HP, BattleLog records and BattleFactory builds. Outside the engine, networking is in the Battle Broadcaster and persistence in the Player Data Client.	A change to the damage formula only touches DamageEffect; a change to the network protocol never touches the engine.
Open/Closed	New action types: a new ActionEffect and a `register()` call. New rules: a new ActionRule in the list. New controllers: a new BattleController. New log events: a new LogEntry subclass. Extra behaviour around the engine: a decorator, like the Battle Broadcaster.	The rules of the game will grow; existing, tested classes do not have to change for that.
Liskov Substitution	Every ActionEffect only changes its target and always returns an EffectResult; every ActionRule only reads and returns a violation or nothing; every LogEntry gives one line of text.	ActionResolver, ActionValidator and BattleLog work with any implementation, without checking the concrete type.
Interface Segregation	BattleEngine is split into BattleActions (controllers) and BattleLifecycle (Battle Coordinator). ActionRule and ActionEffect are small.	A controller cannot call nextTurn() or startBattle(): the compiler prevents it. The complete interface of the assignment still exists as BattleEngine.
Dependency Inversion	BattleEngineFacade receives its collaborators through its constructor. The Battle Coordinator depends on BattleEngineFactory, BattleLifecycle and BattleController instead of classes. ActionEffect is defined in Model and implemented in Effects.	Every part can be tested with a stub, and the Model package does not depend on the concrete effects.

8.2 Design patterns

Pattern	Where	Problem it solves	Alternative considered
Facade	BattleEngineFacade	Other components need six simple methods, not the classes behind them.	Callers use TurnManager, ActionValidator and ActionResolver directly. Rejected: every caller would have to know the order of the steps.
Strategy	ActionEffect with DamageEffect, HealEffect and StatModifierEffect	Three (and later more) kinds of effects, each with its own formula.	One Action class with an enum and a switch in ActionResolver. Rejected: every new type changes the switch.
Strategy	BattleController with the Human and the AI Battle Controller (layer 3)	The turn loop is the same for both players; only who chooses differs.	An if/else on the player kind in the Battle Coordinator. Rejected for the same reason.
Factory	DefaultBattleEngineFactory, BattleFactory, EffectFactory	Building a wired engine, a valid Battle from a ConfigBattle and effects from configuration is complex and belongs in one place.	new everywhere. Rejected: knowledge of how to build objects would be spread over the code.
Decorator	Battle Broadcaster (layer 3)	The client must be updated after every change, without the engine knowing about networking.	The engine sends updates itself. Rejected: it would give the engine a second responsibility.
Singleton	Considered for BattleEngineFacade and rejected	One global engine would be easy to reach from everywhere.	Rejected: it allows only one battle per instance, while up to 8 players can fight at the same time, and global state makes testing hard. The factories are created once per instance by the startup code of the Game Instance, which gives one object without the Singleton pattern.

8.3 Extensibility scenarios

To check that the design is expandable, I went through some likely changes.

Change	What changes	What stays the same
A new action that damages and lowers Defence	Only data: an ActionConfig with a "damage" and a "modifyStat" effect.	All code.
A new effect type, for example "Gravity" (damage = 25% of the target's HP)	A new class GravityEffect and one register() call.	ActionResolver, BattleEngineFacade, validation, log.
A new stat, for example Speed	A new StatType value and a field in UnitConfig.	Stats, effects, rules.
A rule "every unit acts once per turn"	A new class OneActionPerUnitRule; Battle remembers which units acted and TurnManager clears this in passTurn().	ActionValidator, ActionResolver, the interface.
Auto-battle: the AI plays for the player	The Battle Coordinator uses the AI Battle Controller for the human side.	The Battle Engine.
The log in another language	toText() moves to a formatter per language; the entries already store their data.	The data model of the log.

9. Requirements traceability

Requirement	Design element	Section
R-IF1 to R-IF6	BattleEngine (BattleActions + BattleLifecycle), BattleEngineFacade	7.2
R-T1	Battle.activePlayer [1], TurnManager, states HumanTurn and ComputerTurn	7.3, 7.8
R-T2	Battle owns exactly 2 BattlePlayers; PlayerKind HUMAN and COMPUTER	7.3
R-T3	Human and AI Battle Controller outside the engine	6.1, 6.4
R-T4	TurnManager.passTurn()	7.2, 7.10
R-T5, R-A1	CasterRule (NOT_ACTIVE_PLAYER)	7.5
R-A2	Action (casterId, targetIds), ActionDefinition, TargetCountRule, TargetSelectionRule	7.4, 7.5
R-A3, R-U4	DamageEffect	7.4, 7.10
R-A4, R-U5	HealEffect	7.4, 7.10
R-A5, R-A6	StatModifierEffect, Stats.applyPercentage()	7.4, 7.10
R-U1, R-U3	Unit.takeDamage(), Unit.heal()	7.3, 7.10
R-U2	Unit.isKnockedOut, BattlePlayer.knockedOutUnits, CasterRule, TargetSelectionRule	7.3, 7.5, 7.9
R-S1	ConfigBattle, Battle.createSnapshot()	7.3, 7.6
R-L1	BattleLog and the LogEntry subclasses	7.7
R-C1, R-C2, R-C3	Instance Manager, InstanceRoutingApi	5.1, 5.4
R-C4	Instance Agent and Instance Manager (shutdown after 30 seconds)	5.4, 6.1
R-C5	Instance Manager and Player Session Manager, maxPlayers = 8	5.4, 6.3
R-C6	Persistence Service, Player Data Client (write-through)	5.5, 6.5
R-C7	Game Client on the Player PC	5.1, 5.2
R-C8	ConfigBattle (1..4 units per player), BattleFactory validation	7.6

10. Design Justification

10.1 Design process

The design went through three versions: two design iterations and a change of technology.

Version 0.1 (23 September 2026) contained the goal, a context diagram with one person and one system, a use case diagram, a deployment diagram and a component diagram of the containers, and a first layer 3 diagram with a session controller, an AI controller, the battle engine and a persistence client.

Review (7 October 2026). I compared version 0.1 with the assignment and the assessment criteria. The main findings:

- The diagrams did not use the right notation: the use cases were rectangles with arrows and no system boundary, the deployment diagram had no UML nodes or artifacts, and the "component diagram" was really a container diagram without interfaces.
- There were no diagrams in C4 notation, and the names differed between views (for example Persistence server and Persistence service).
- The instance rules of the case (max. 8 players, private instance, switching, shutting down) were not visible in the design.
- The goal said that the engine knows nothing about the rest of the game, while the layer 3 diagram let it call persistence.
- Layer 3 had no boundary and no interfaces, and layer 4, the data model, the explanation of the rules, SOLID and the justification were missing.

Version 1.0 (8 October 2026) solves these points:

Finding	Change in version 1.0
Wrong UML notation	All UML diagrams redrawn: system boundaries and ovals, nodes with stereotypes and artifacts, components with provided and required interfaces.
No C4 notation	A C4 diagram with a legend for layers 1 to 3; purple marks the element the next layer zooms in on.
Inconsistent names	One name per element in all views, and a mapping table (section 5.5).
Instance rules missing	Responsibilities of the Instance Manager, a sequence diagram for visiting a world, and a deployment specification with maxPlayers.
Engine calls persistence	The Battle Coordinator saves the results; the engine has no dependencies (DN-3.7).
Layer 3 incomplete	Game Instance boundary, all components, interfaces, a detailed deployment diagram and two sequence diagrams.
Layer 4 missing	Package diagram, seven class diagrams, a state machine, the applyAction sequence and the implementation of the rules.
No justification	Interpretations (3.3), design notes per layer, SOLID and patterns (chapter 8) and this chapter.

Some names changed as well: the session controller of version 0.1 is now the Human Battle Controller, the AI controller is the AI Battle Controller and the persistence client is the Player Data Client.

Version 1.1 (8 October 2026) changes the technology from C# and .NET to Java 21 (decision D13). Java is the language used in this course, so the design can be implemented with the knowledge and tools of the course. The structure of the design did not change. The names now follow Java conventions: methods in camelCase, interfaces without an "I" prefix (the interface is BattleEngine and the facade class BattleEngineFacade), jar files instead of assemblies and Spring Boot instead of ASP.NET Core. Java has no equivalent of the C# "internal" visibility, so encapsulation of the battle now relies on the snapshot: the live Battle object never leaves the engine (section 7.3).

10.2 Design decisions

ID	Decision	Alternatives	Why this choice
D1	The Battle Engine is a library inside the Game Instance.	A separate battle service.	A battle belongs to one player on one instance; a service would add a network call per action and a second place with state.
D2	The engine has no dependencies on networking or persistence.	The engine saves results itself (version 0.1).	Matches the goal and keeps the engine testable and reusable; the Battle Coordinator decides what to save.
D3	One BattleEngineFacade object per battle.	(a) One engine per instance with a battleId on every method. (b) A Singleton engine.	(a) changes the given interface: nextTurn() and getBattleState() have no battleId. (b) allows only one battle per instance. With one object per battle the interface stays exactly as given; the battleId of getBattleLog() is checked against the battle of the engine.
D4	getBattleState() returns a deep copy.	(a) The live Battle object. (b) Separate DTO classes for every model class.	(a) lets callers change the battle without the rules. (b) doubles the number of classes. A deep copy of the model is safe (the live objects never leave the engine) and simple.
D5	Action (request) is separate from ActionDefinition (data).	One Action class with everything, including the damage value.	The available actions do not have targets yet, and numbers sent by a client cannot be trusted.
D6	Effects as Strategy objects, 1..* per action.	An enum with a switch; a subclass of Action per type.	New effect types without changes to existing classes, and combinations of effects without new code.
D7	Validation as a list of rule objects.	One large validate() method with if-statements.	Every rule is small and testable, the order is explicit and new rules can be added.
D8	Knocked-out units move to a separate list.	Delete them, or only set a flag.	Deleting makes the state incomplete; with only a flag every query must filter. A separate list does both.
D9	The Battle Coordinator owns the turn loop; controllers via BattleController; BattleEngine split in two.	The controllers call nextTurn() themselves.	One place decides about turns, and the compiler prevents a controller from passing a turn.
D10	Stat rounding with integer arithmetic.	Floating point and Math.ceil().	Floating point gives wrong results for values like 10 × 1.1.
D11	Write-through cache in the Player Data Client.	Save when the player leaves.	No progress is lost when an instance stops or crashes.
D12	WebSocket between client and instance.	UDP; HTTP polling.	Server push in both directions and it works through firewalls; a turn-based battle does not need UDP.
D13	Java 21 for all containers: jMonkeyEngine for the client, Spring Boot for the services.	C# and .NET with Unity (version 1.0).	Java is the language of this course. One language for client and server means shared data classes for the network messages.

10.3 Insights

- **Notation is part of the design.** My first use case and component diagrams showed the right ideas with the wrong symbols, so they said something different from what I meant. Using the real UML elements (boundaries, interfaces, nodes) forced me to make the design more precise.
- **The given interface hides a design question.** nextTurn() and getBattleState() have no battleId, while an instance can have up to 8 players in a battle at the same time. Thinking about this led to one engine object per battle and to the Battle Coordinator.
- **A small engine makes the rest easier.** Keeping networking and persistence out of the engine gave the turn loop, the updates to the client and the saving of results each their own clear place.

- **Two classes for actions solved two problems.** Separating what a unit can do (ActionDefinition) from what it does now (Action) lets getAvailableActions() return actions without targets, and makes sure the server never trusts numbers from the client.
- **Checking with examples finds real problems.** Testing the design against the examples of the assignment (section 7.10) showed that the obvious implementation of rounding up, with floating point numbers, gives wrong results.